IoT Communication Protocols and Messaging

LIAU, Yee Yeng Ph.D.

2026

Fall Semester

IoT Communication Protocols and Messaging

Bridge from Week 4

Week 3

Prepare the data

↓

Week 4

Establish connectivity

↓

Week 5

Communicate the data

Sensor

↓

IoT device

↓

Wi-Fi / Ethernet / Cellular / etc.

↓

Network available

Now, we **need to decide:**

Connectivity exists

↓

Where the data should go?

↓

Which application/process should receive it?

↓

How should data be transported?

↓

How the application should exchange messages?

Week 5

Journey of IoT data **from a connected device** to an **application**

IoT Communication Protocols and Messaging

Learning objectives

- Explain IP communication (network protocol) in IoT
- Explain transport protocols: TCP and UDP
- Explain request–response and publish–subscribe
- Describe application protocol: HTTP, CoAP, and MQTT
- Trace IoT data through the protocol stack
- Select suitable communication protocols

Contents

1. **IP and Transport**
   - Where the data goes and how it is transported?
2. **Request-Response Communication**
   - How an application asks for or modifies a resource?
3. **MQTT and Publish-Subscribe**
   - How to handle many continuously producing devices and consumers of their data?

1. IP and Transport

Communication stack

One protocol cannot do everything

❖ Once a device is connected → other functions

Layer	Examples	Main question
Connectivity / Link	Wi-Fi, Ethernet, IEEE 802.15.4	How does the device access the network?
Network protocol	IP	Where should the data go?
Transport protocol	TCP, UDP	Which application should receive it? How should it be delivered?
Application protocol	HTTP, CoAP, MQTT	How should applications exchange information?

Wi-Fi → connectivity

↓

IP → destination addressing

↓

TCP → transport

↓

MQTT → application-level messaging

➢ **Different layers solve different communication protocols**

1. IP and Transport

Communication stack

One protocol cannot do everything

Consider package delivery:

- **Network protocol** = university and building location (host)
  - ✓ IP → which host?
- **Transport protocol** = room number (application/process)
  - ✓ Port → which application / process
- **Application protocol**
  - ✓ HTTP / CoAP / MQTT → rules governing content and interaction

- **IP** finds the device
- **TCP/UDP** delivers data to the correct application
- **HTTP/CoAP/MQTT** define how applications exchange meaningful messages

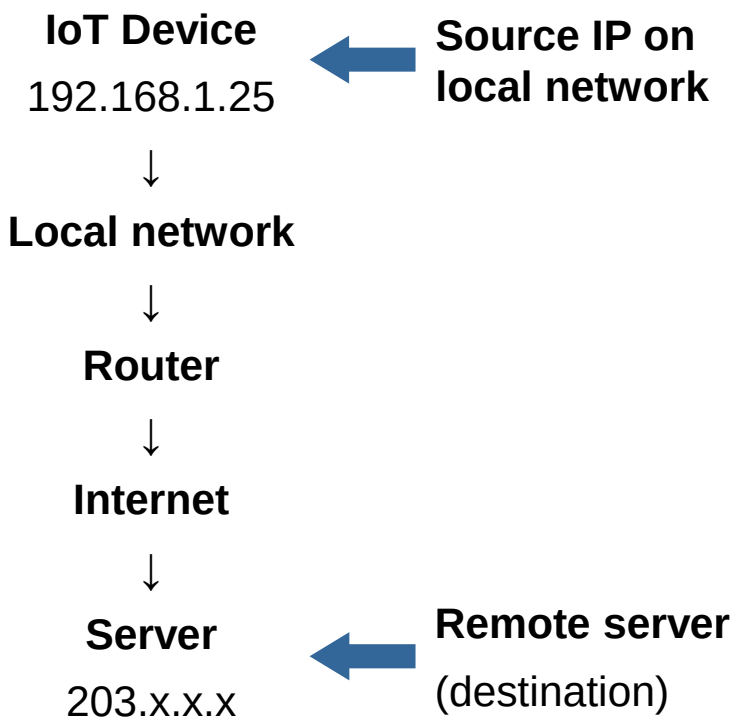
1. IP and Transport

IP address

- **IP** address (Internet Protocol address)
  - Logical address used to identify a device / interface on an IP network

- **Main purposes:**
  - Identify the **source**
  - Identify the **destination**
  - Allow data to be delivered **across interconnected networks**

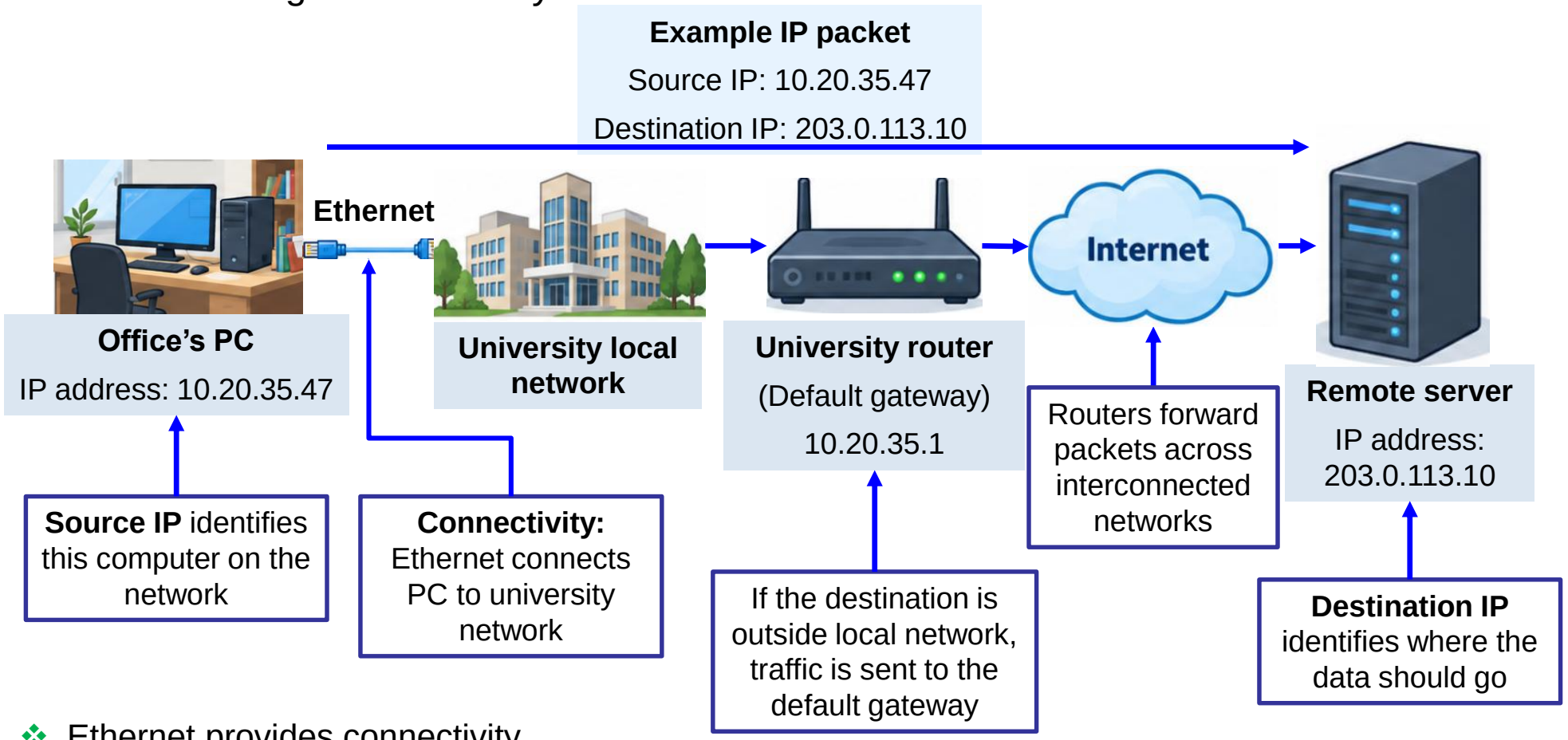
- **Communication technology** establishes **access to a network**
- **IP** enables communication **across networks**



1. IP and Transport

IP addressing

- **IP** addressing in a university network



- Ethernet provides connectivity
- IP addressing identifies source and destination across network

1. IP and Transport

IPv4 address

- **Example: 192.168.1.25**
- **IPv4** uses 32 bit address
- Usually written as 4 decimal numbers

192 . 168 . 1 . 25

↓ ↓ ↓ ↓

8 bits 8 bits 8 bits 8 bits

11000000.10101000.00000001.00011001

→ full Ipv4 address

192 = 11000000

168 = 10101000

1 = 00000001

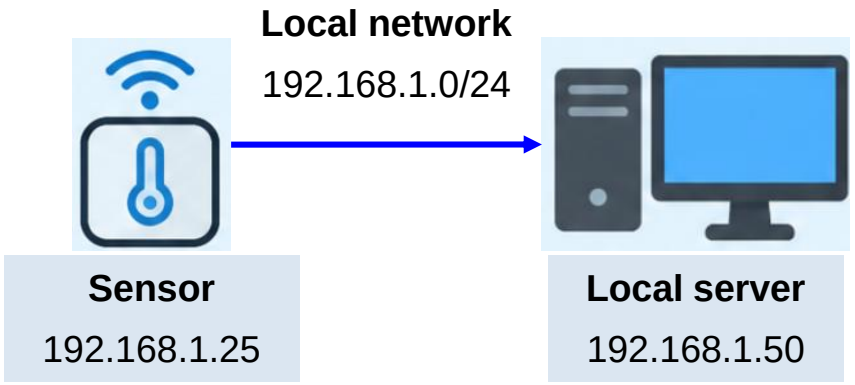
25 = 00011001

- Identifies a network interface locally
- Address may change depending on network configuration

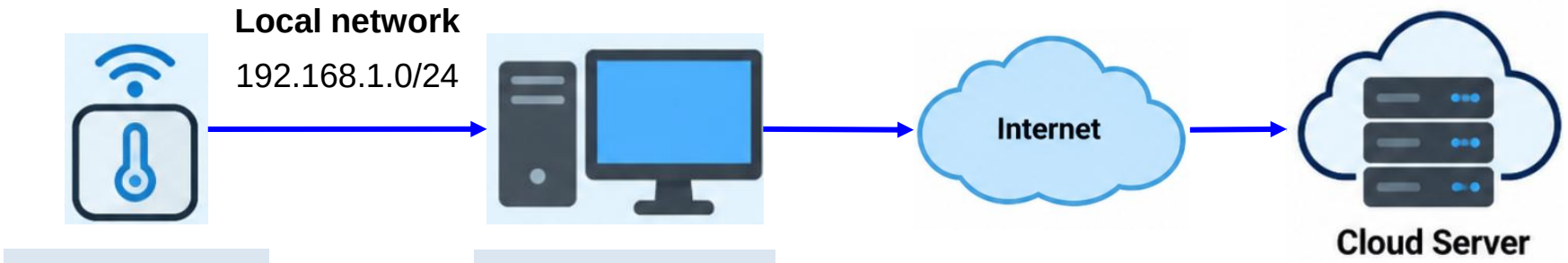
1. IP and Transport

Local communication vs remote communication

❖ Case A: Local communication



❖ Case B: Remote communication

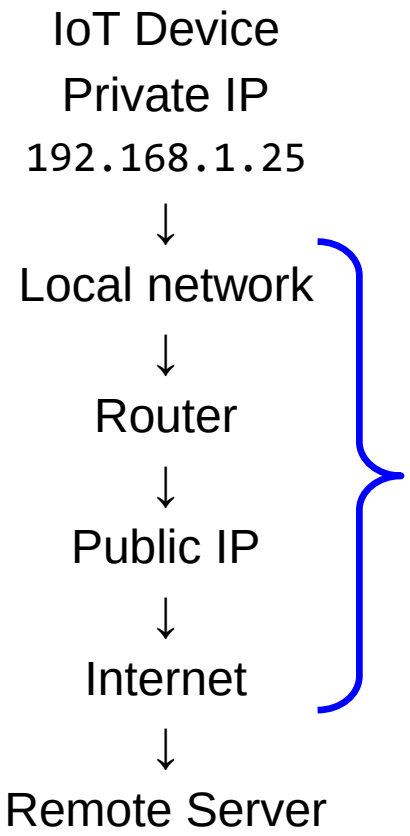


Router forwards IP traffic between **different networks**

1. IP and Transport

Private and public address

❖ **Conceptually:**



Private IP	Public IP
Used within local networks	Used for communication on the public Internet
Not directly routable on public Internet	Globally routable
Example: 192.168.1.25	Assigned to Internet-facing network/interface

Router connect **local(private) network to internet (public)**

uses a **public IP** address for internet communication

1. IP and Transport

NAT – Network Address Translation

Consider:

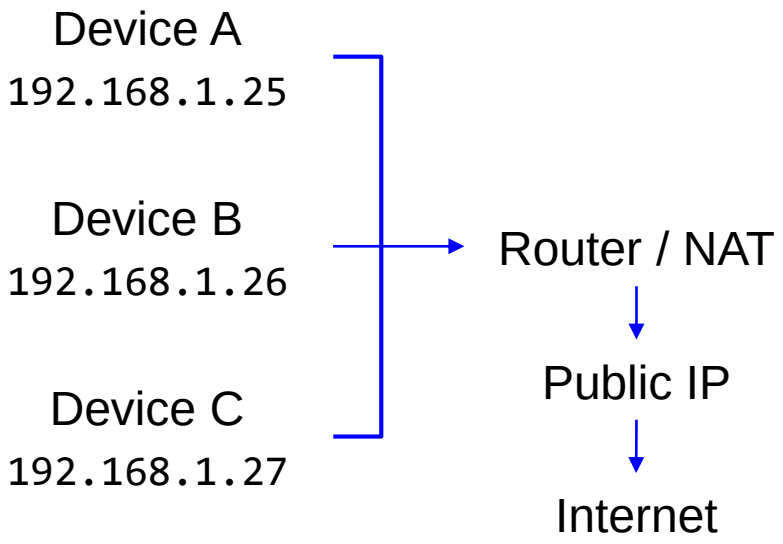
- *Many devices inside the local network have private addresses,*
  - *but they still want to communicate with Internet services*

→ **NAT – used at network boundary**

- **NAT translates addresses between a private network and an external network**

Why NAT is useful:

- Many local devices can use private IP addresses
- They can still communicate with Internet services
- Helps conserve public IPv4 addresses



- **With NAT, computers in a home, office, or university can use private IPv4 addresses while still accessing the Internet**

1. IP and Transport

IPv6 address

- **Example: 2001:db8:abcd:0012::25**
- **IPv6** uses 128 bit address

2001 : 0db8 : 0000 : 0000 : 0000 : ff00 : 0042 : 8329

↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓

16 b 16 b 16 b 16 b 16 b 16 b 16 b 16 b

1. Leading zero can be removed
2. Consecutive zero → ::

   2001:db8::ff00:42:8329

4 hexadecimal digits × 4 bits per hexadecimal digit = 16 bits

8 groups

→ Total = 128 bits

- Provides larger address space than IPv4
  - Does not mean every IoT device must use IPv6 directly
- Designed to support continued Internet growth
  - Large-scale IoT can involve very large numbers of networked devices

1. IP and Transport

IPv4 vs IPv6

	IPv4	**IPv6**
Address size	32 bits	128 bits
Format	Decimal, dotted	Hexadecimal, colon-separated
Example	192.168.1.25	2001:db8::25
Address space	Limited	Extremely large

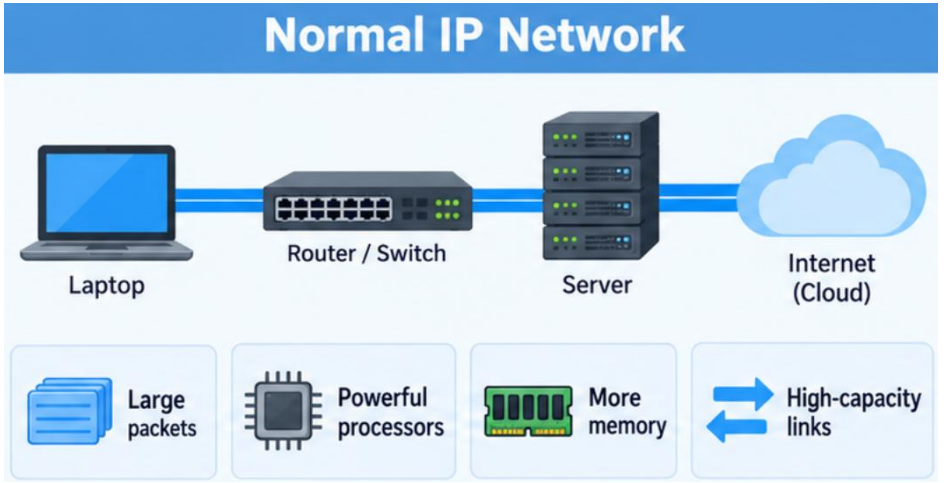
- **IPv6** does not automatically solve every IoT networking problem
  - → Problems still exist on constrained devices:
    - Battery limitations
    - Small memory
    - Limited processing capability

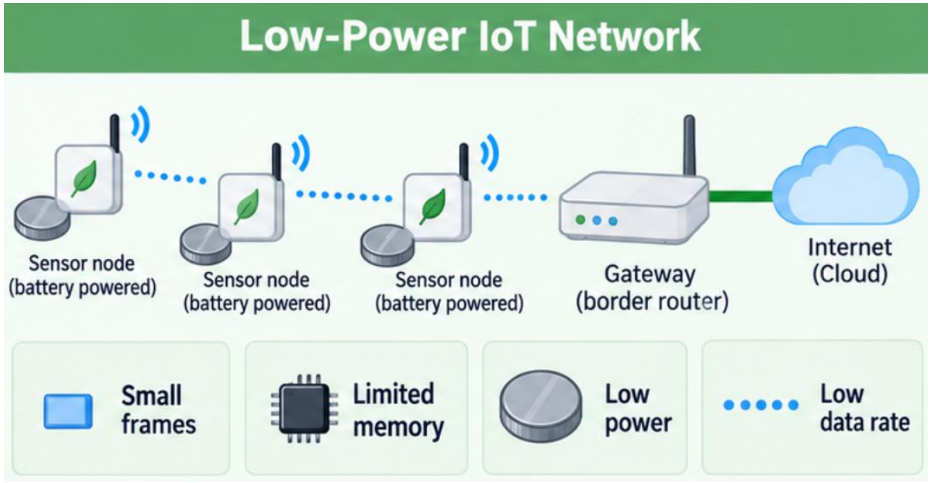
NAT, IPv6, IPv4, Public vs Private IP

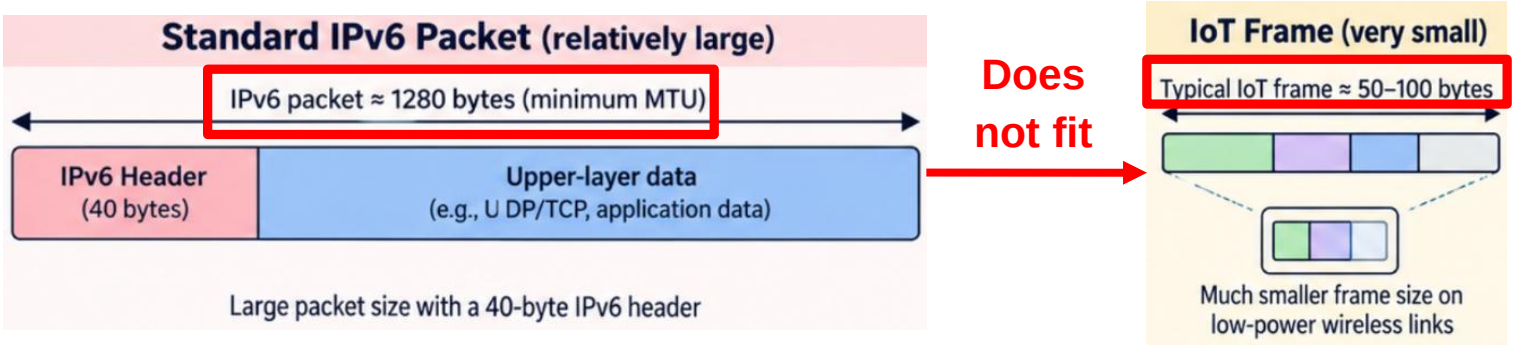
https://www.youtube.com/watch?v=CXReQUkCZ0w

1. IP and Transport

Constrained network problem







- Standard IPv6 communication may **create significant overhead** on constrained device and low-power links

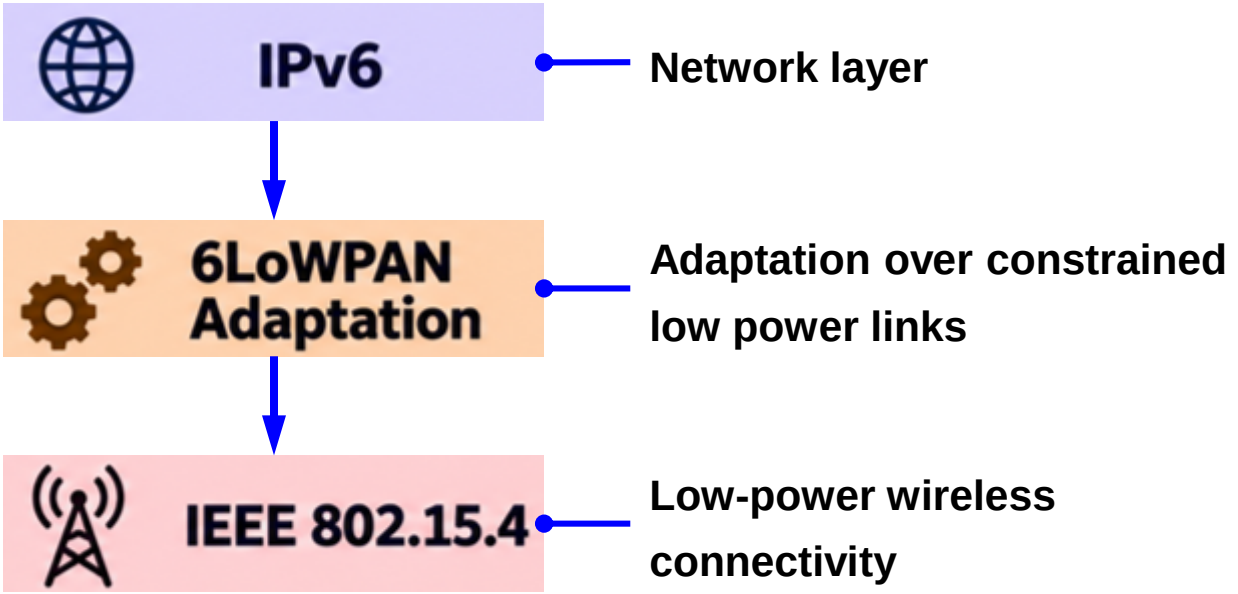
Requires adaption

→ **6LoWPAN**

1. IP and Transport

6LoWPAN

- **6LoWPAN – IPv6 over Low-Power Wireless Personal Area Networks**
- An adaptation layer that:
  - Enables IPv6 packets to be transmitted over low-power, constrained wireless network

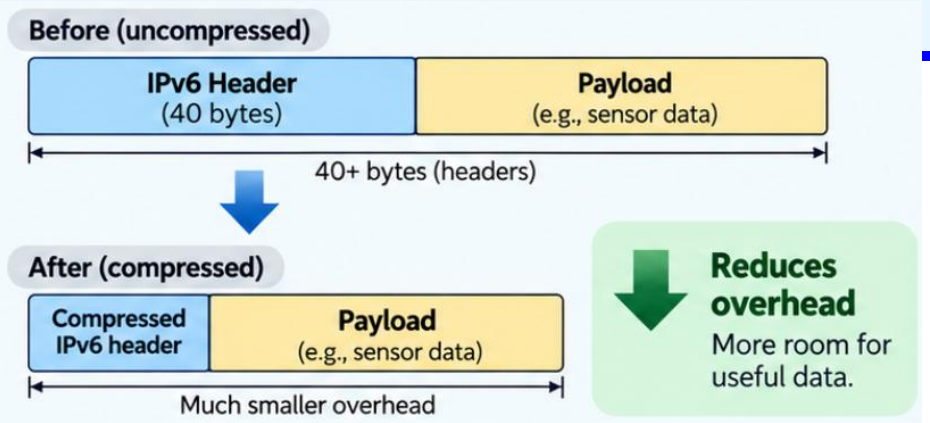


1. IP and Transport

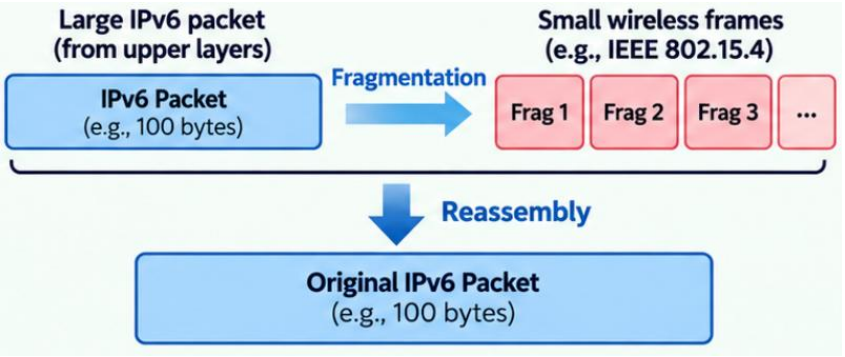
6LoWPAN

Key functions of 6LoWPAN:

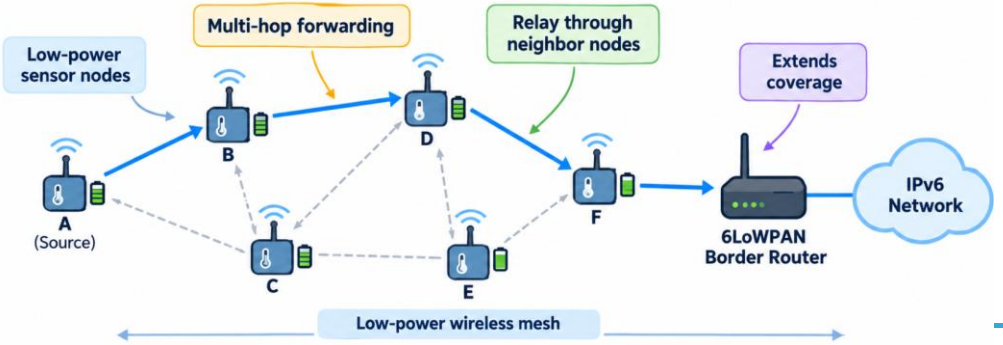
- **Header compression**
  - **Omitting redundant header fields** to reduce standard 40-byte IPv6 headers to as few as 2 to 7 bytes



- **Fragmentation & Reassembly**
  - Before transmission, **splits large IPv6 packets into smaller fragments**
  - **Reassembles** them at the receiving node

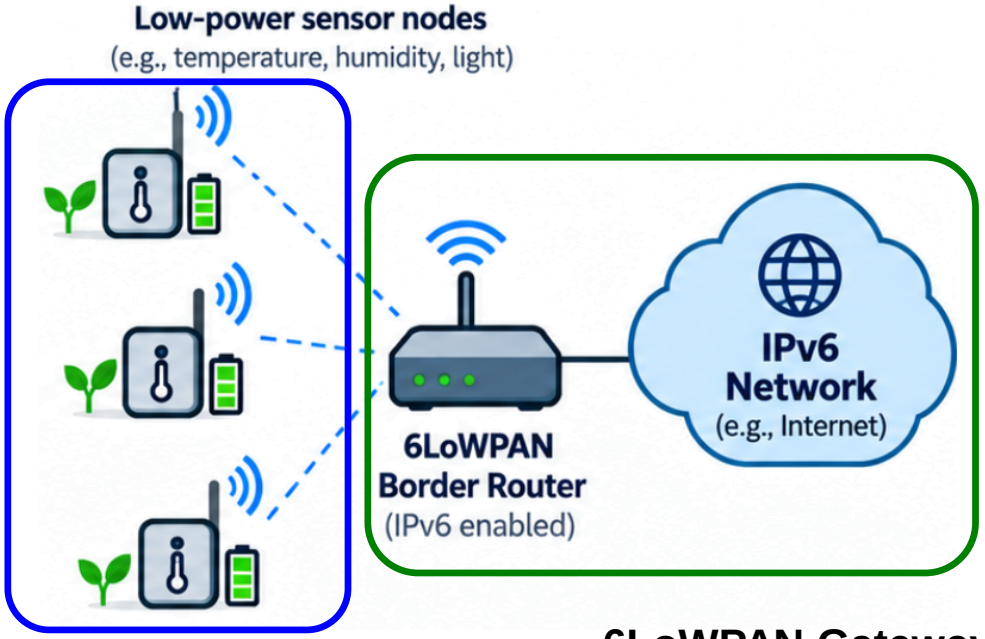


- **Mesh routing adaptation**
  - Supports link- or network-layer routing across **low-power wireless mesh topologies**



1. IP and Transport

6LoWPAN



Low-Power IoT Sensors (6LoWPAN Nodes):

- Communicate through constrained wireless network
- To collect and transmit environmental data using low-power radios

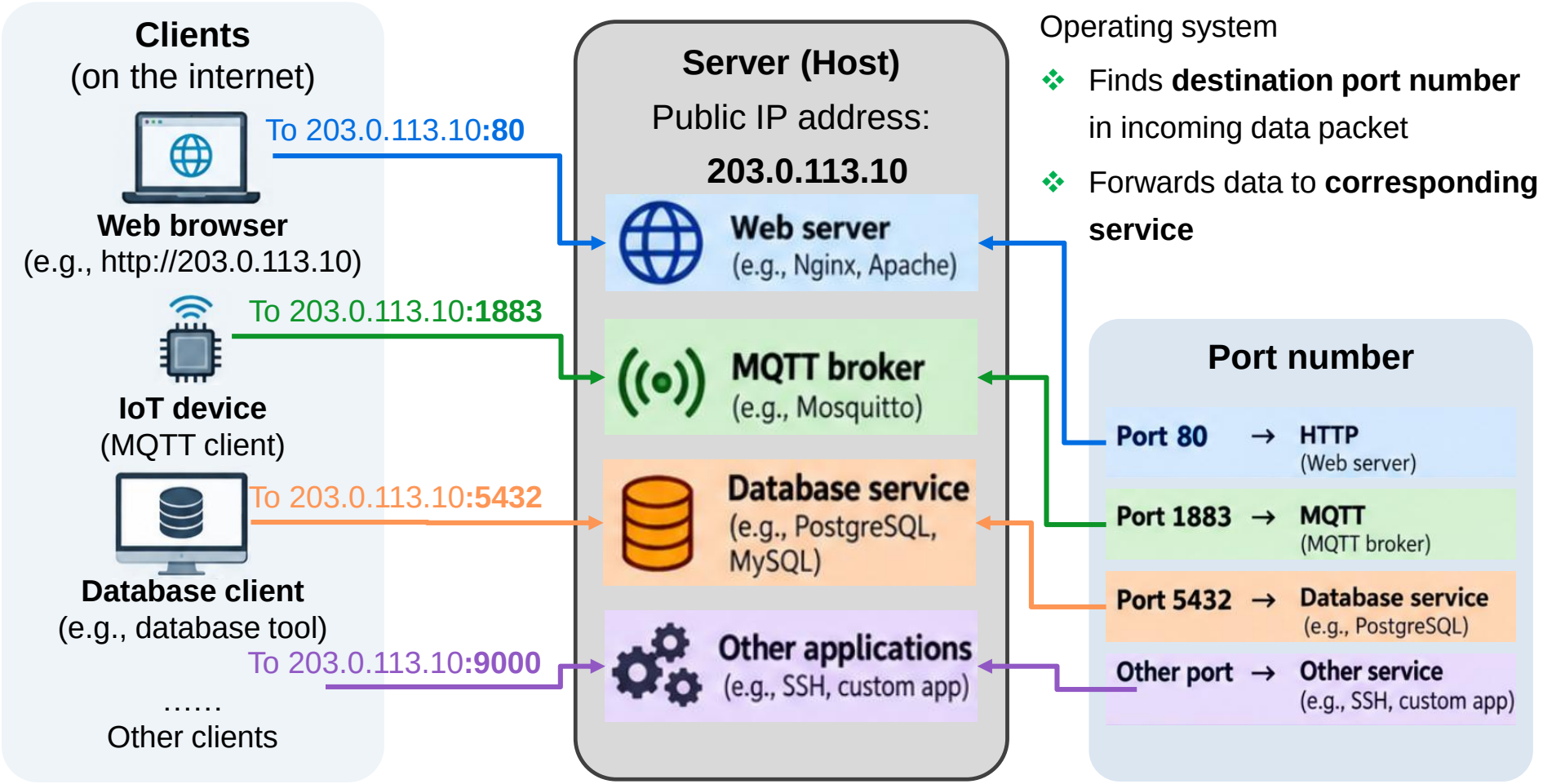
6LoWPAN Gateway / Border Router:

- **Connects** the low-power wireless mesh network to the global IPv6 Internet
- Once the packet enters wide IP network → continue using ordinary IPv6 network

1. IP and Transport

Ports and transport

❖ A **single machine (host)** can run many services, all share the same IP address



1. IP and Transport

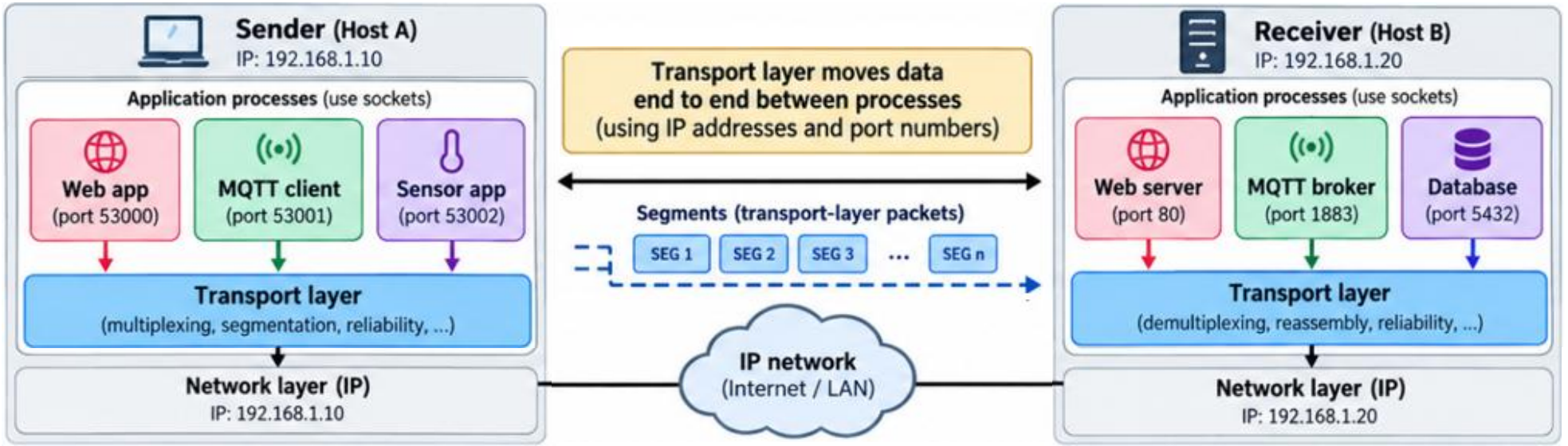
Transport layer

❖ Transport layer

- ➢ Provides **process-to-process (end-to-end) communication**
  - Between application processes running on different host across an IP network
- ➢ Ensures data reached specific software application running on target host

❖ Network layer

- ➢ Handles **host-to-host routing**

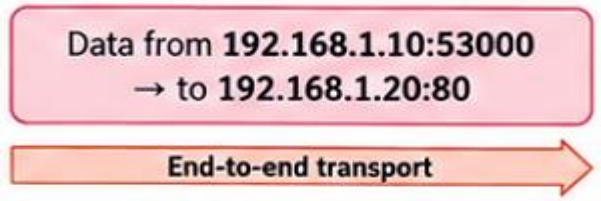


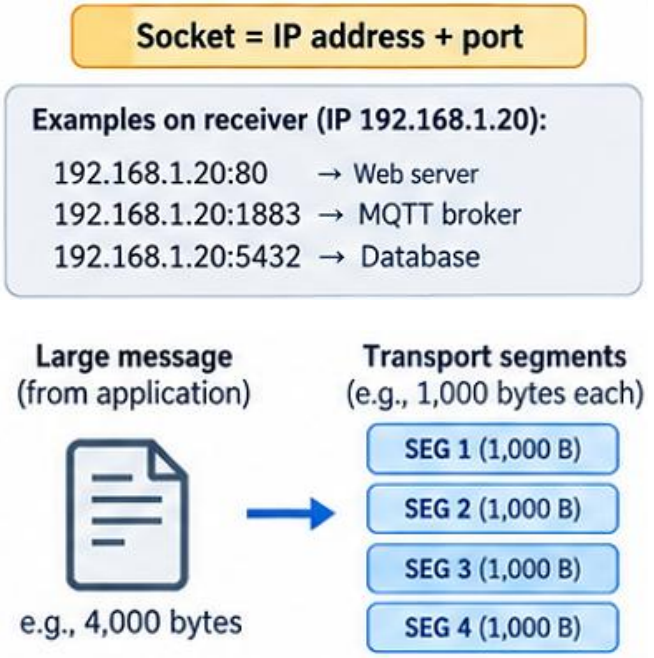
1. IP and Transport

Transport layer

❖ Functions of transport layer

- **Application multiplexing** through ports
  - Transport layer uses port numbers to **identify** which application process data belongs to
- **Segmentation** of data
  - **Splits** large application messages into smaller transport-layer segments
- **Delivery behavior**
  - Provides **end-to-end, process-to-process delivery** using IP addresses and port numbers
  - **Delivers** data to correct application process at destination host



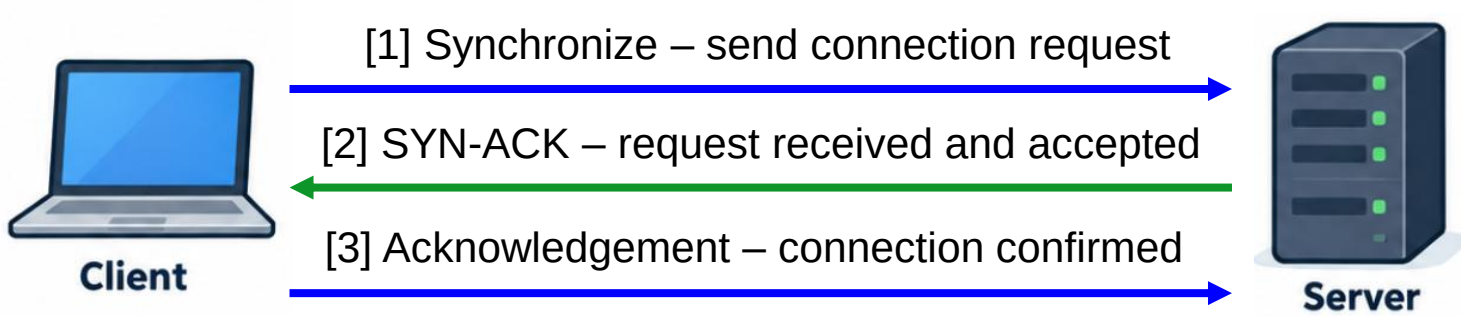


1. IP and Transport

TCP (Transmission control protocol)

❖ TCP (Transmission Control Protocol)

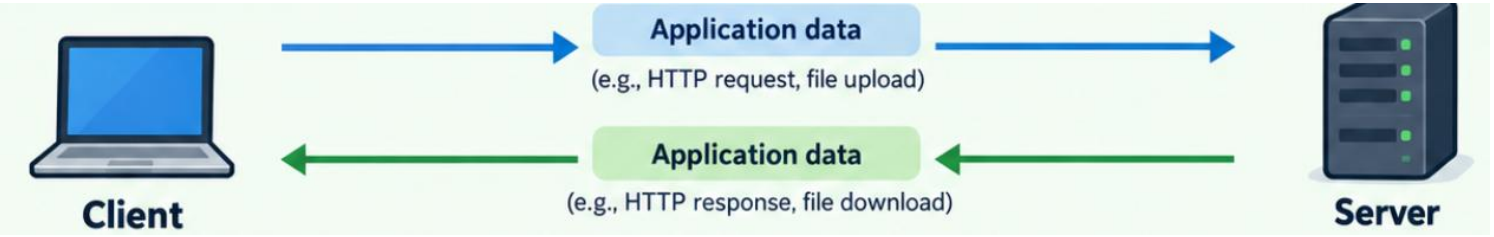
- **Creates a logical connection ("handshake")** before application data is exchanged



❖ Why a connection exists:

- Communication state established at both endpoints
- Sequence numbers coordinated for ordered delivery

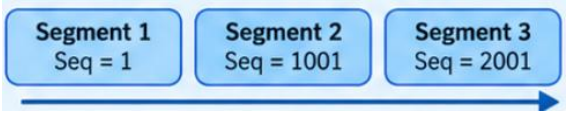
❖ After connection setup



1. IP and Transport

TCP (Transmission control protocol)

❖ Four ideas in TCP:



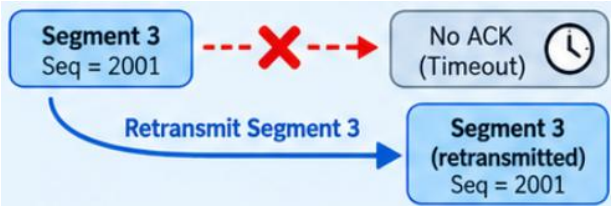
1. **Sequence/order**
   - TCP numbers data as segment – placed in correct order

2. **Acknowledgment**



   - Receiver confirms received data
   - ACK number – next expected byte (cumulative ACK)

3. **Retransmission**
   - If expected ACK does not arrive, sender retransmits

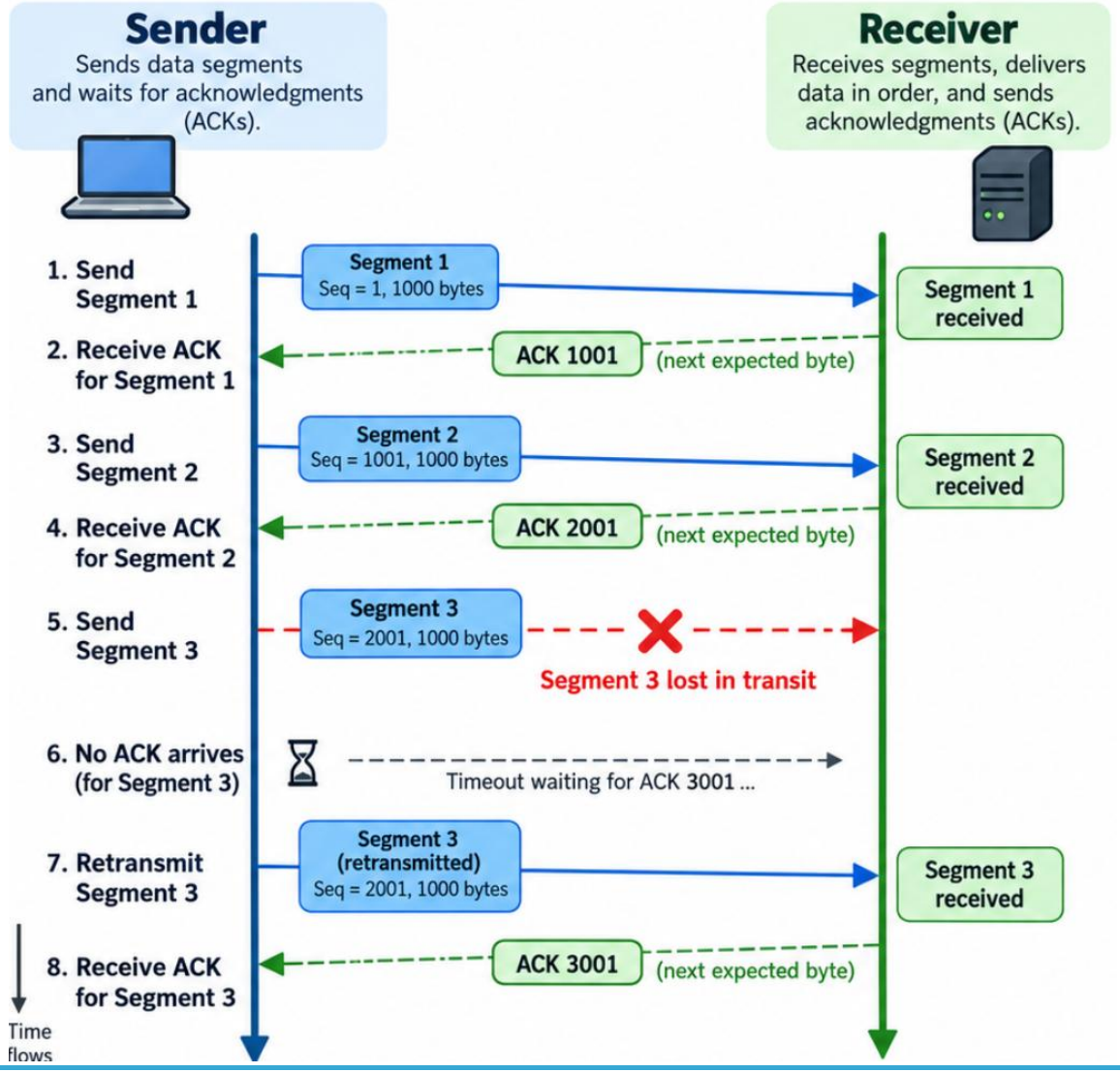


4. **Duplicate handling**
   - If duplicate data arrives, TCP can detect and ignore duplicates



1. IP and Transport

TCP (Transmission control protocol)



Sawyer Cobot Communication Via TCP/IP

https://www.youtube.com/watch?v=1L-ZU7Rw08M

1. IP and Transport

UDP (User Datagram Protocol)

❖ **UDP**

- ➢ Simple, high speed communication protocol
- ➢ Sends data across internet without establishing connection or checking if data arrives

Characteristics:

❖ Connectionless

- No handshake to establish connection

❖ Unreliable delivery

- Does not track whether data packets are lost, delayed or duplicated

❖ No ordering

- Packets can take different routes across network

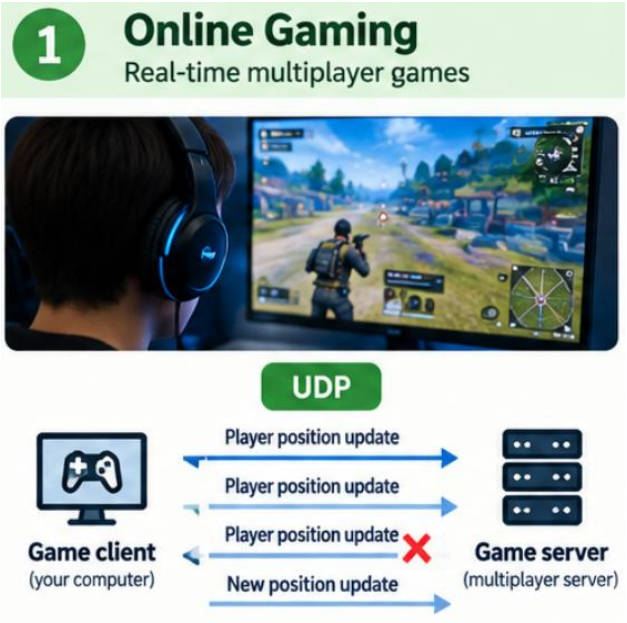
❖ No flow control

- Sends data as fast as application generates it
- Even receiving device is congested to handle

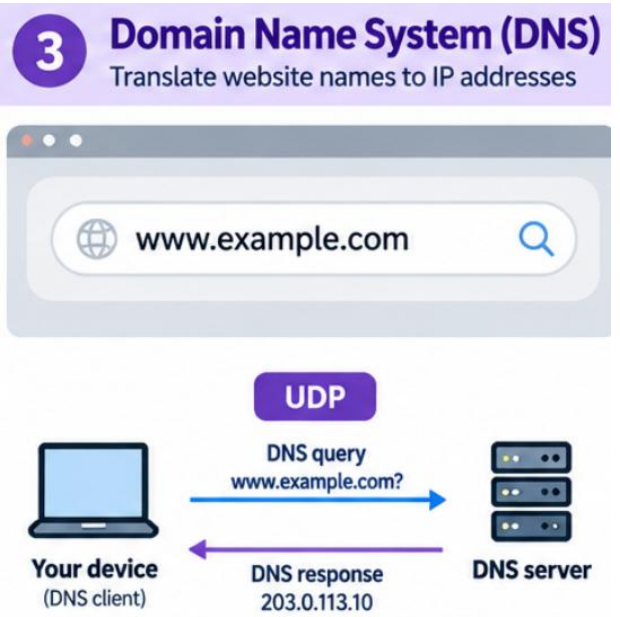
1. IP and Transport

UDP (User Datagram Protocol)

❖ Common real-world uses







TCP VS UDP

https://www.youtube.com/watch?v=WAxip2eSxn4

2. Request-Response Communication

Opening

Transport protocol

TCP or UDP moves data,

but how does an application **express**

“give me the temperature” or “change this setting”?



Application protocols

Define **rules and interaction** for application-to-application communication,

so devices can

→ request resources (“give me temperature”)

→ change settings (“set the interval”)

2. Request-Response Communication

Opening

❖ **Transport alone is insufficient**

➢ TCP does not understand data (measurement)

Example:

❖ TCP can deliver: 101010001101…

❖ But TCP does not know:

- whether it means temperature;
- whether it is a request;
- whether it is a command;
- which resource is being requested

Transport layer

TCP / UDP

Delivers application data to the correct process

↓

Application protocol

HTTP / CoAP / MQTT

Defines **how applications interpret** and **exchange messages**

❖ **Application protocol gives context to the exchange**

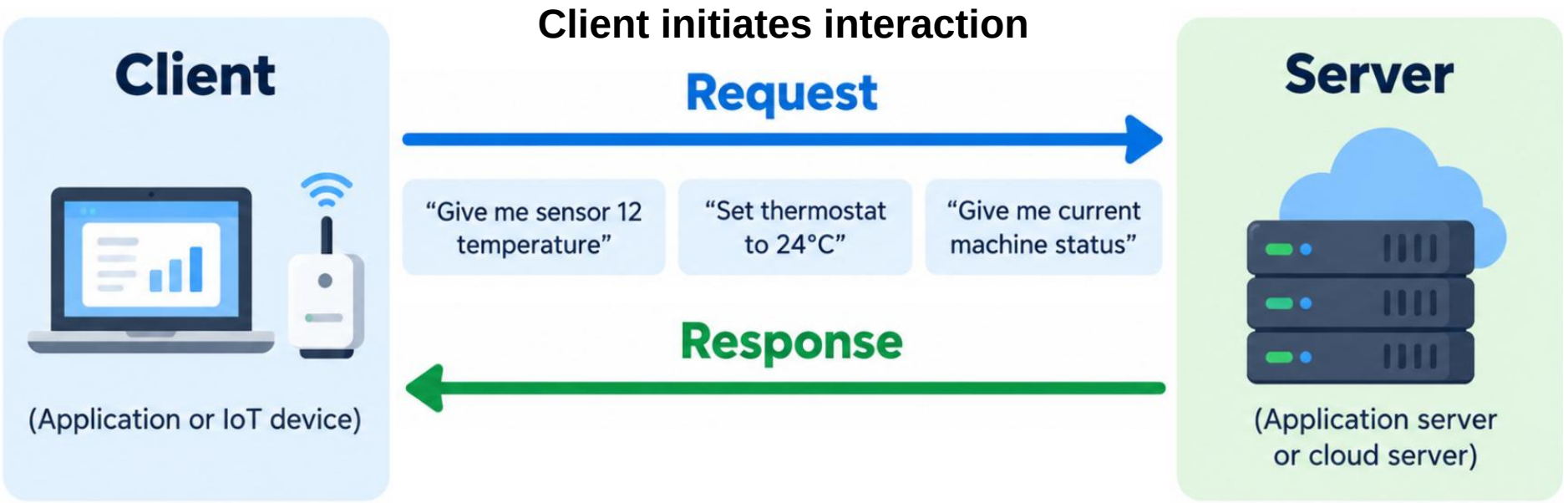
➢ HTTP: `GET /api/sensors/12/temperature` → retrieve resource

➢ CoAP: `GET /temperature` → tells application it is a request

2. Request-Response Communication

Request-response

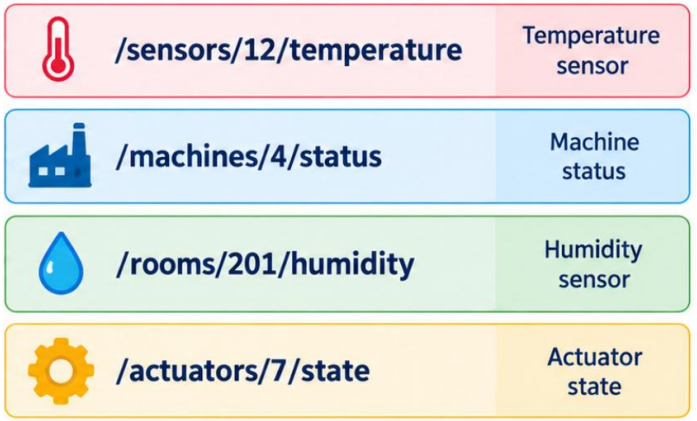
❖ Request-response model



2. Request-Response Communication

Resource-oriented communication

- **Resource-oriented communication** structures application data and hardware functions into:
  - Distinct, uniquely addressable paths (URIs)



Example:

- Client requests a specific resource

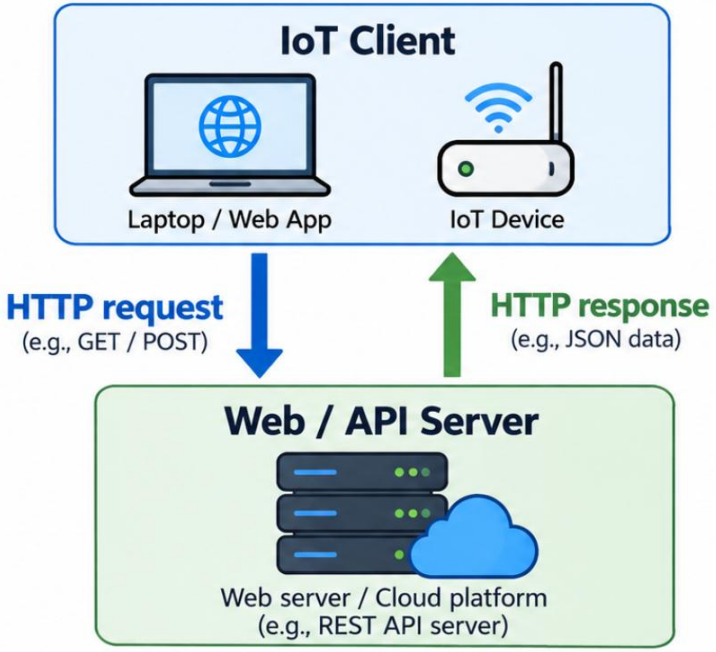


2. Request-Response Communication

HTTP architecture in IoT

HTTP (Hypertext Transfer Protocol)

- Application-layer protocol running over TCP/IP (transport layer)
- Structures data exchange between a client and a server through a **request-response model**



IoT / Web Client

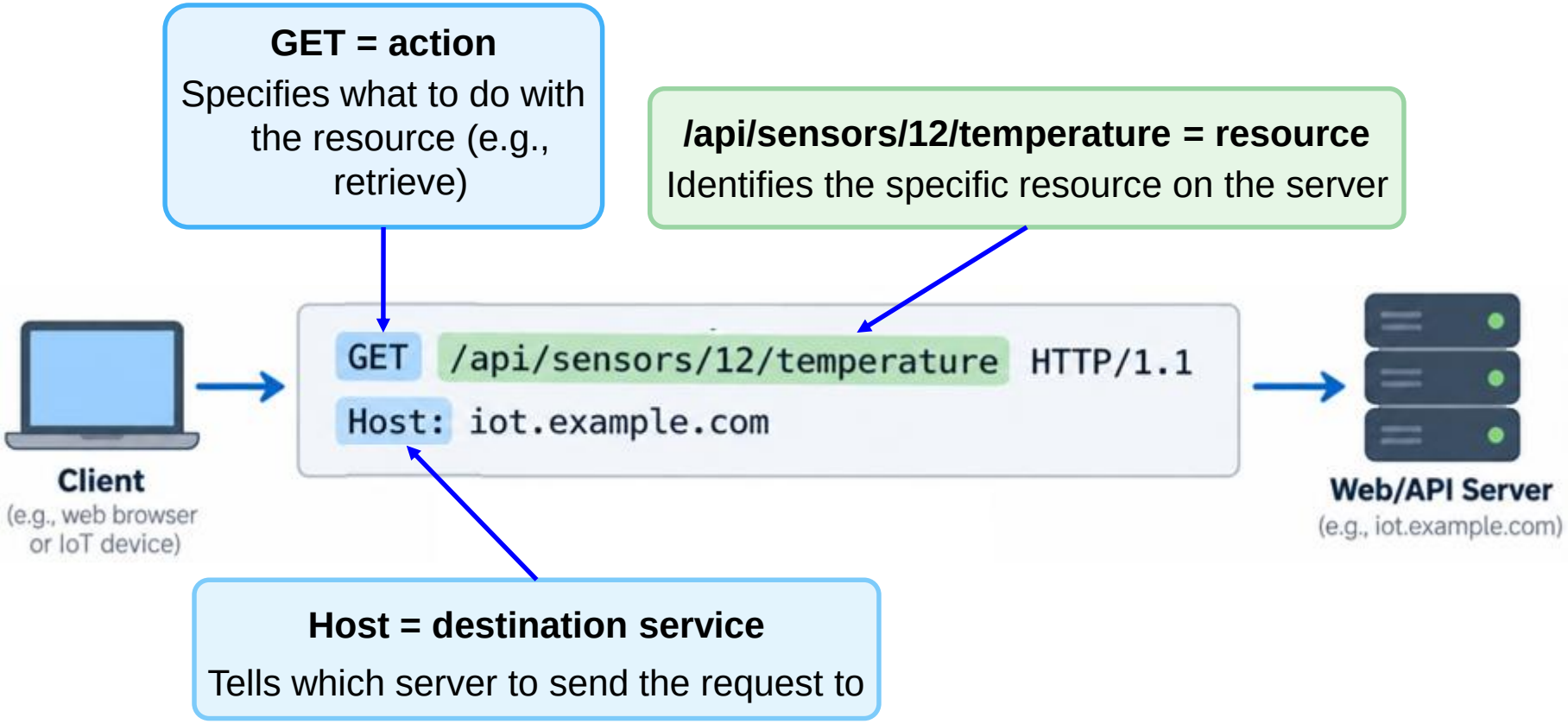
- Initiates interaction by establishing a TCP connection and sending an HTTP request

Web / API Server

- Processes the incoming request
- Executes the requested action
- Returns an HTTP response containing a status code and payload

2. Request-Response Communication

Anatomy of HTTP request



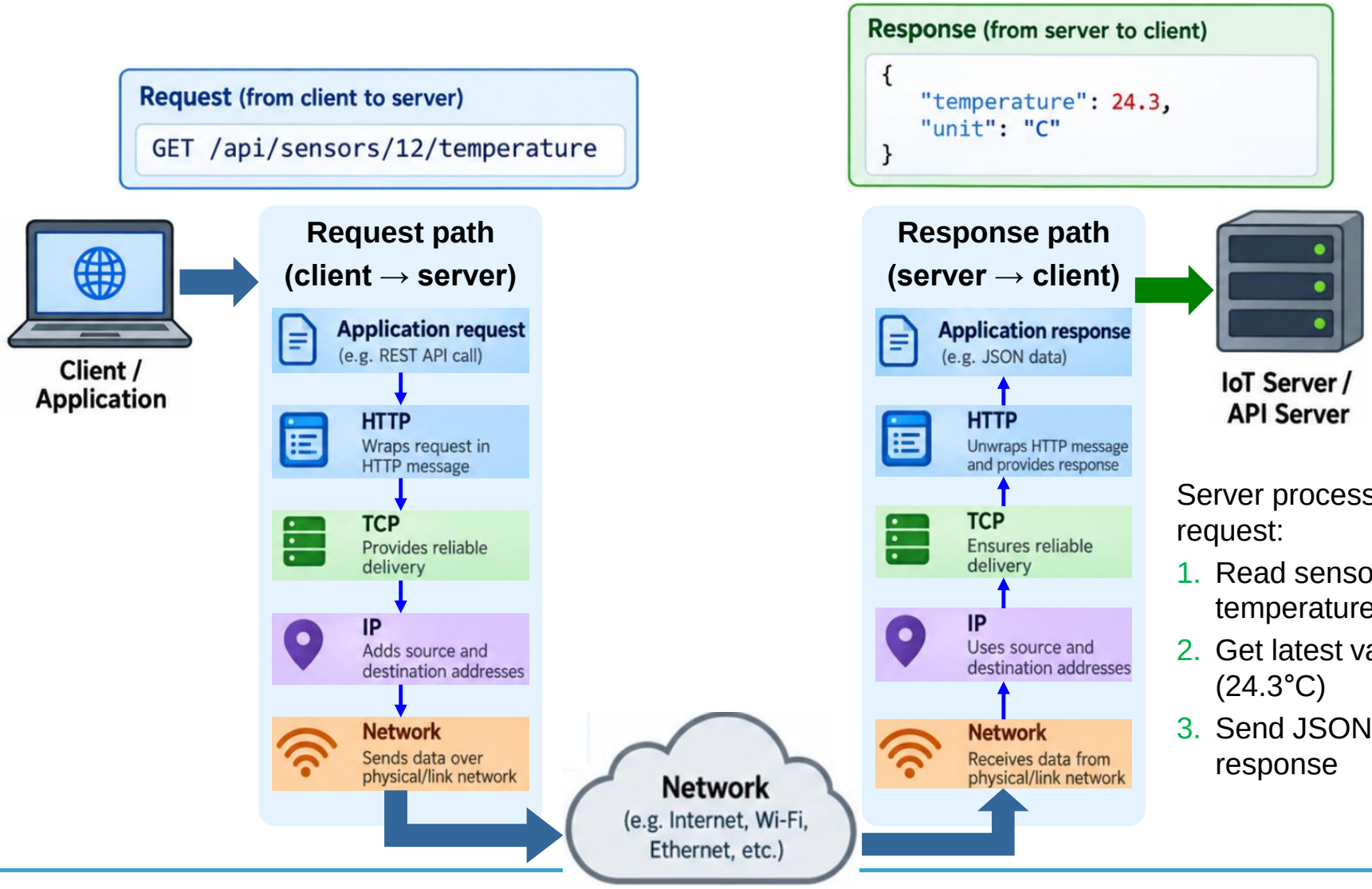
2. Request-Response Communication

Common HTTP methods

Method	Meaning in IoT example
GET	Read sensor status
POST	Submit data/create action
PUT	Update a resource/configuration
DELETE	Remove a resource/configuration

2. Request-Response Communication

HTTP Example – Reading sensor data in server



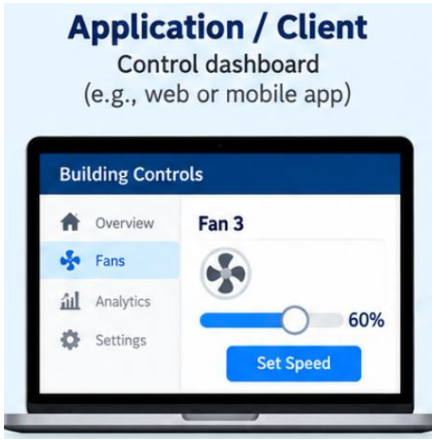
Server processes request:

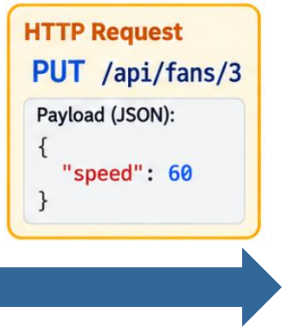
1. Read sensor 12 temperature
2. Get latest value (24.3°C)
3. Send JSON response

2. Request-Response Communication

HTTP Example – Actuator command

Application sends a command to configure a device (fan)



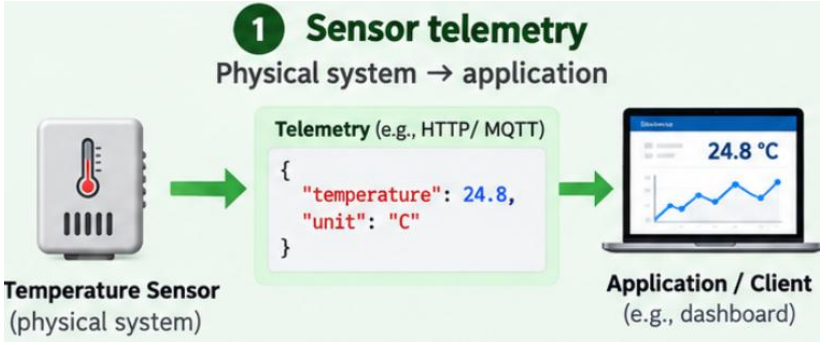


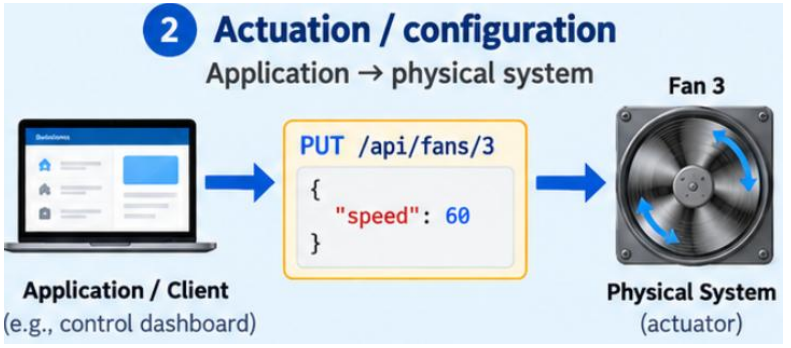




- **Receives** the command
- **Authenticates & validates**
- **Forwards** to the device
- **Updates** device configuration

Two information flow directions in IoT





2. Request-Response Communication

HTTP status codes

Code	Status
200	OK
201	Created
400	Bad request
404	Not found
500	Server error

Example:

Request: `GET /sensors/99/temperature`

But, sensor 99 does not exists

Server: 404 Not found

HTTP & HTTPS Protocol

https://www.youtube.com/watch?v=HyK2tu4nFRM

2. Request-Response Communication

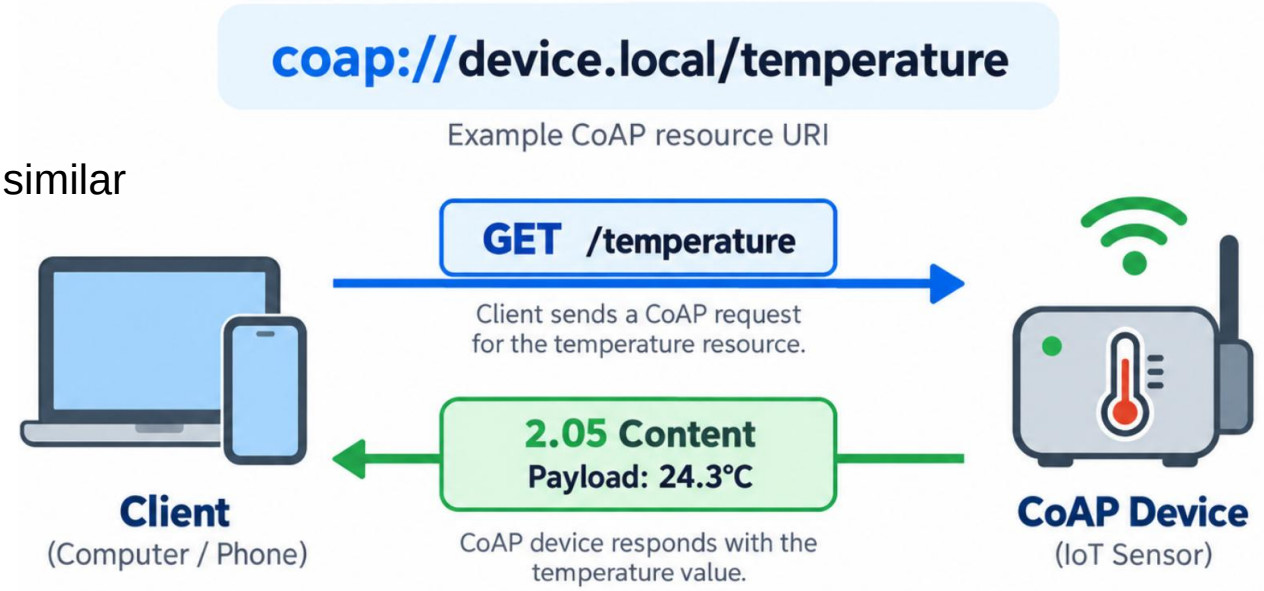
CoAP (Constrained Application Protocol)

CoAP (Constrained Application Protocol)

- Lightweight application protocol for **constrained IoT environments**

Characteristics

- Lightweight
- Resource-oriented
- **Request–response model** similar conceptually to HTTP
- Commonly uses **UDP**



2. Request-Response Communication

HTTP vs CoAP

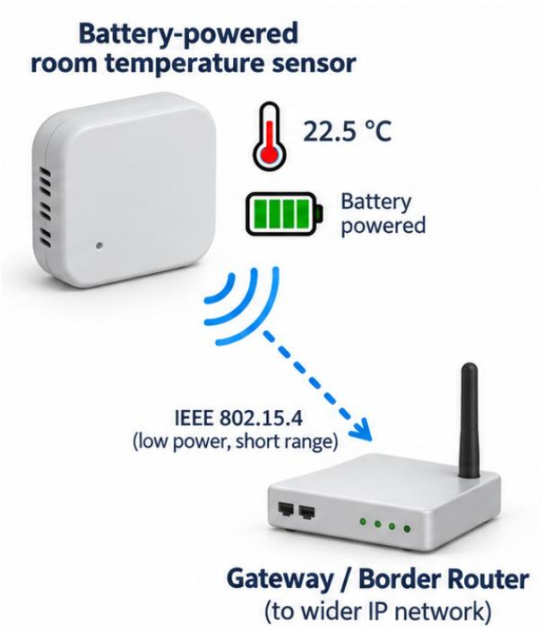
	HTTP	CoAP
Interaction model	Request–response	Request–response
Typical transport	TCP*	UDP commonly
Message representation	Text-oriented HTTP messages	Compact binary
Typical environment	Web/cloud/API systems	Constrained IoT
Resource model	Yes	Yes

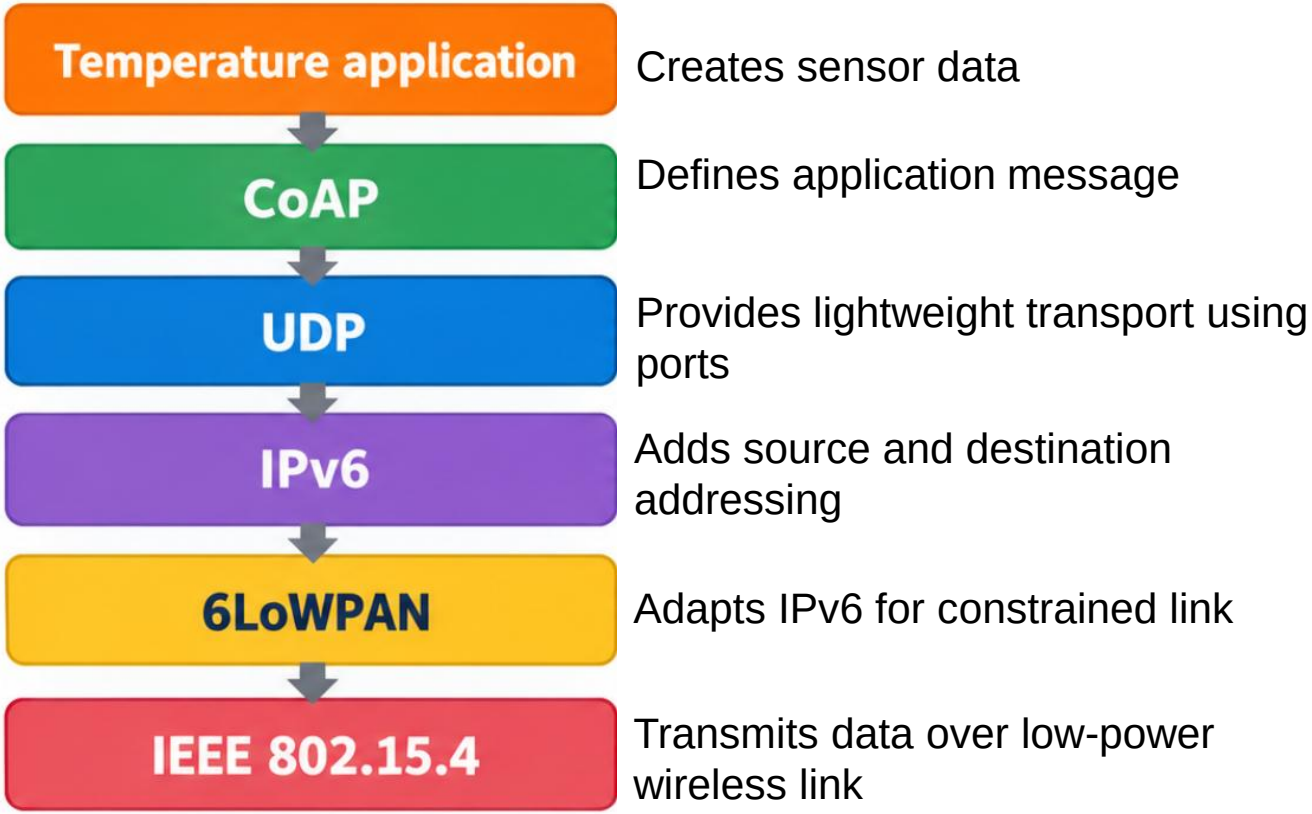
2. Request-Response Communication

CoAP example

Constrained room sensor

→ Outgoing data path from a battery-powered room-temperature sensor

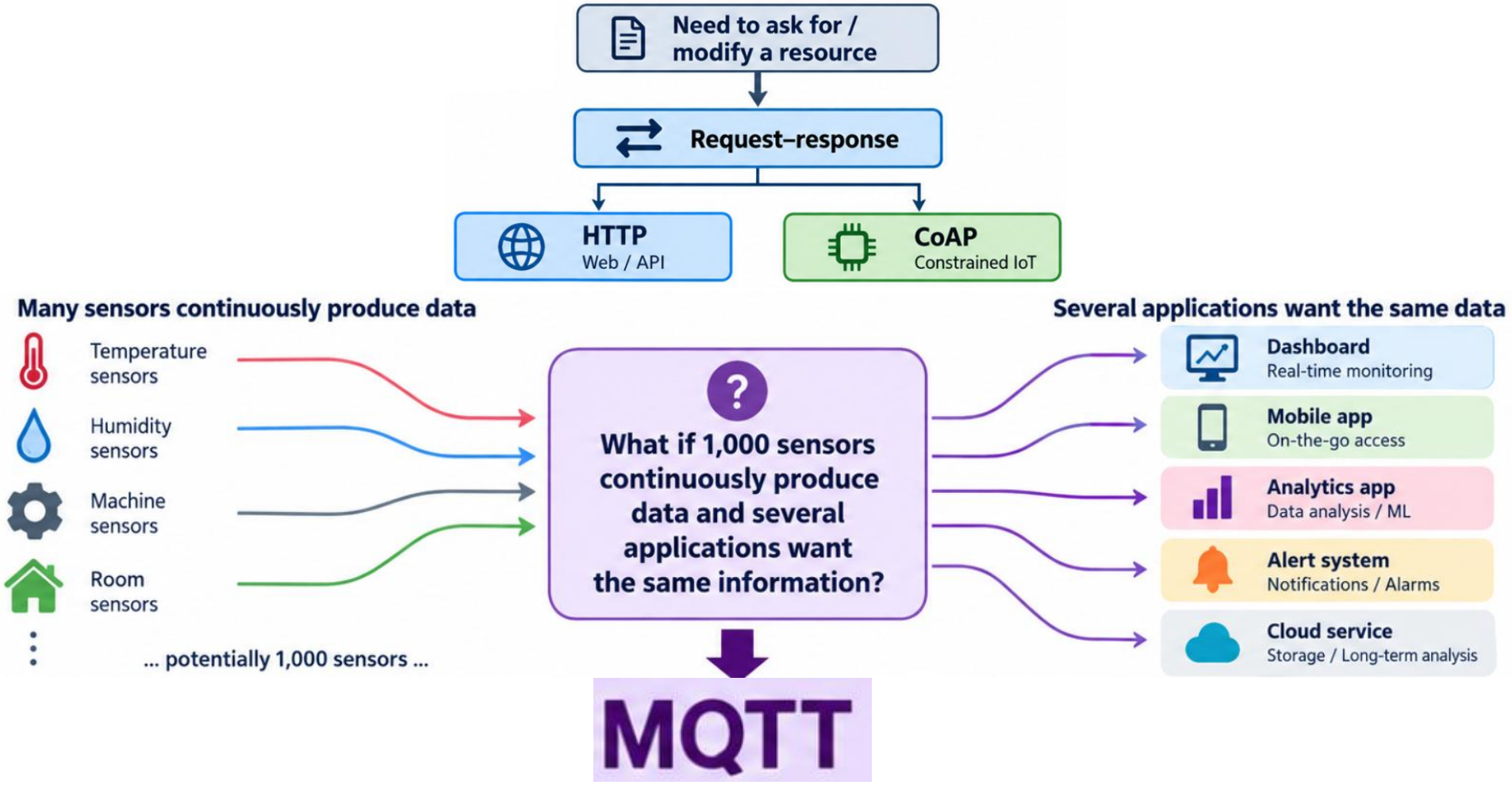




→ Incoming data is processed through the stack in the reverse direction

2. Request-Response Communication

Summary

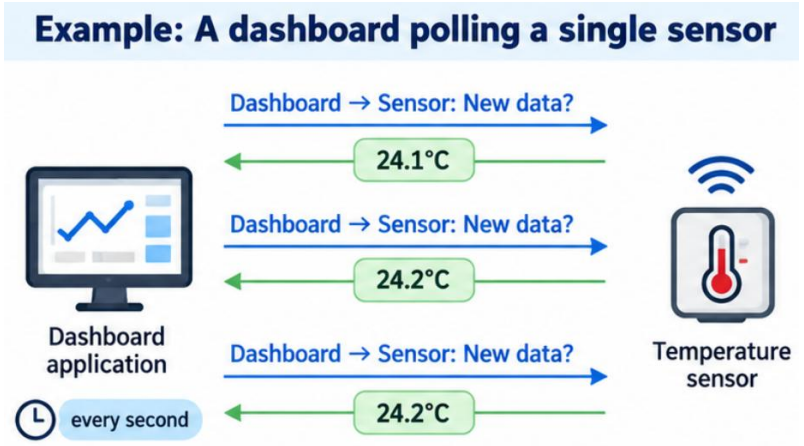


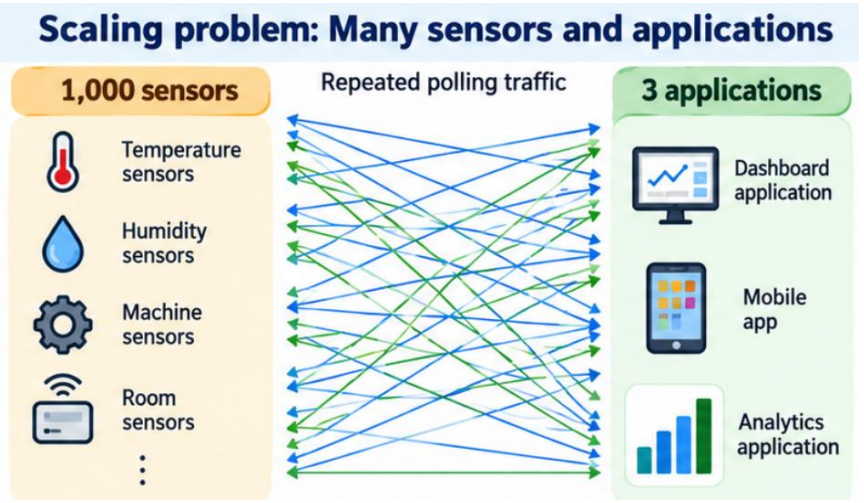
3. MQTT and Publish-Subscribe

Problem of request-response

Polling problem

- A system continuously asking a device or resource
  - if it has new data to share, even when nothing is happening
- Example: keep walking to front door to check if the package is there → exhausting





When **many sensors and applications** are involved, polling can create:

- Repeated requests
- Unnecessary traffic
- Poor scaling

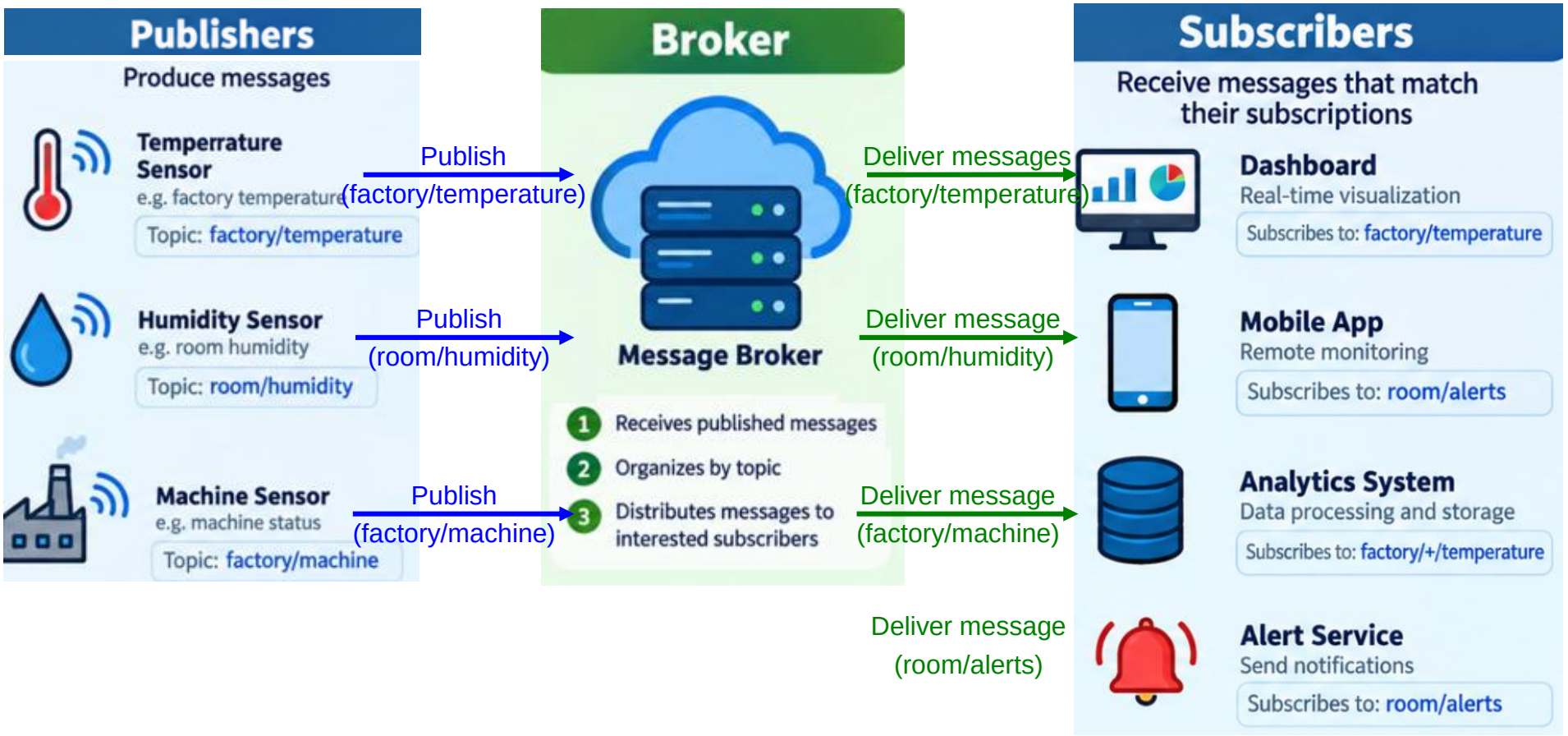
3. MQTT and Publish-Subscribe

Publish-subscribe model

[1] Publisher sends message to a topic

[2] Broker receives and routes message by topic

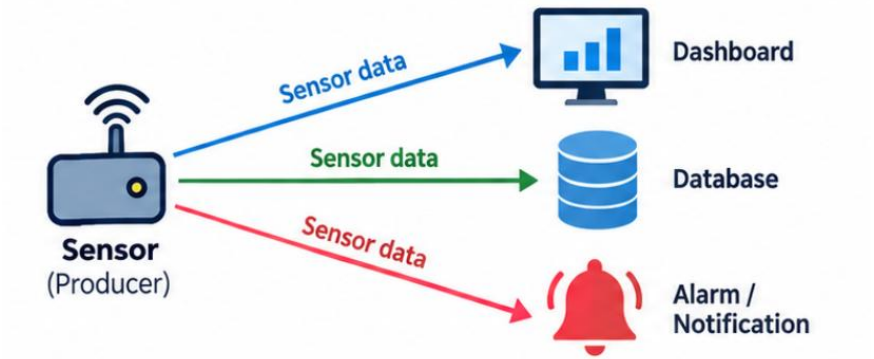
[3] Subscriber receives matching topic



3. MQTT and Publish-Subscribe

Decoupling in IoT messaging

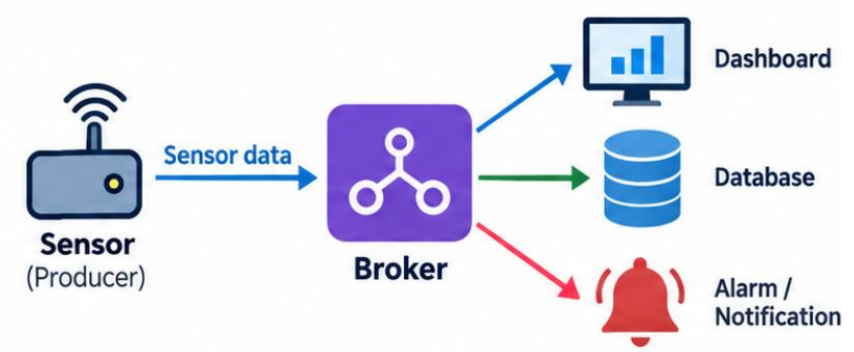
Direct communication



Sensor must know all destinations

- Tighter coupling
- More direct connections
- Hardee to scale when applications increase

Brokered communication



Sensor communicates mainly with broker

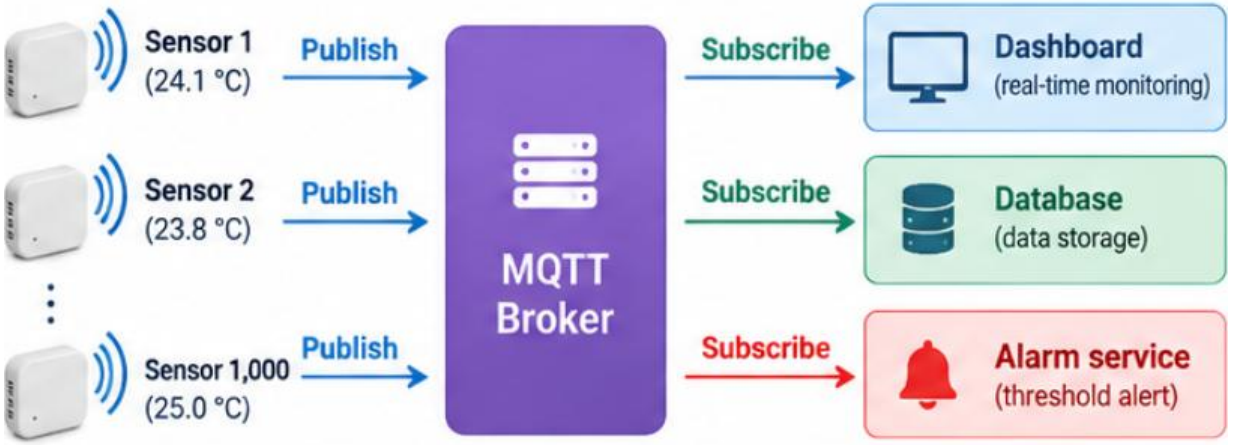
- Decoupling between sensors and applications
- Easier distribution
- Adding new applications is simpler

- **Broker-based messaging decouples (separate) sensors and applications**

3. MQTT and Publish-Subscribe

MQTT architecture

- **MQTT (Message Queuing Telemetry Transport)**
  - Light-weight publish-subscribe messaging protocol used in IoT
- **Sensors publish data** to a **broker**
- **Applications subscribe** to data they are interested in



Publisher sends message to topic → Broker receives and routes message → Subscribers receive messages for subscribed topic

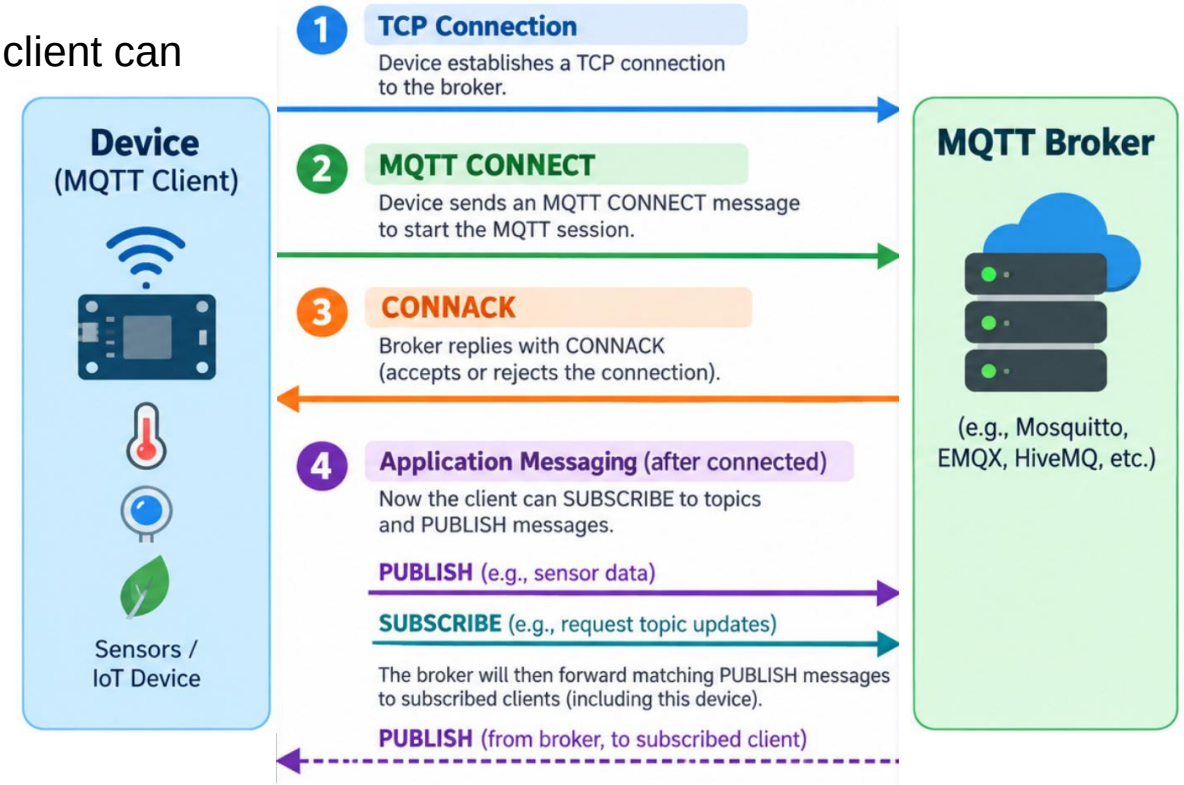
- Publishers and subscribers do not need to know each other directly

3. MQTT and Publish-Subscribe

MQTT connection and message flow

❖ **MQTT typically operates on top of TCP/IP**

1. Device establishes a **TCP connection** to broker
2. Client **sends CONNECT message** to start MQTT connection
3. Broker replies **CONNACK**
4. **Application messaging begins**, client can
   - ➢ PUBLISH messages and
   - ➢ SUBSCRIBE to topics



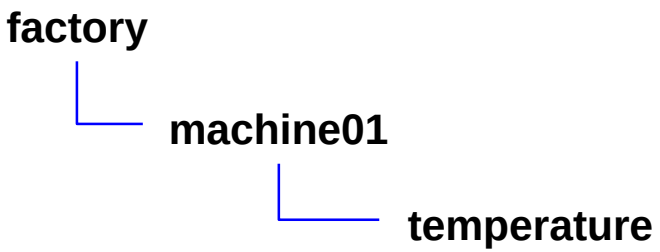
3. MQTT and Publish-Subscribe

MQTT topics

❖ **Example:**

- factory/machine01/temperature
- factory/machine01/vibration
- factory/machine02/temperature

Hierarchy:



Design useful topic

❖ Poor topic:

× Sensor1

× Sensor2

× Data3

❖ **Better topic:**

– factory/line1/machine01/temperature

– factory/line1/machine01/vibration

❖ **Good topic design** provides:

✓ **understandable** hierarchy

✓ **scalable** organization

✓ **easier** subscription

✓ **separation** of measurement types

3. MQTT and Publish-Subscribe

MQTT wildcards

- **Wildcards are used in subscription topic filters**
- Suppose a factory has following topics:

factory/machine01/temperature
factory/machine01/vibration
factory/machine01/status

factory/machine02/temperature
factory/machine02/vibration
factory/machine02/status

factory/machine03/temperature
...

Without wildcards:

- A dashboard interested in every machine's temperature would need to subscribe separately

  factory/machine01/temperature
  factory/machine02/temperature
  factory/machine03/temperature
  ...
  factory/machine100/temperature

Types of wildcards:

1. single-level wildcard
2. Multi-level wildcard

3. MQTT and Publish-Subscribe

MQTT wildcards – single level "+"

Single-level wildcard: "+"

- "+" matches exactly one topic level

Example:

- `factory/+/temperature`
- Break into levels:

```
factory / + / temperature
          ↑
   Exactly one level
```

- **Match:**

`factory/machine01/temperature`

`factory/machine02/temperature`

`factory/machine99/temperature`

→ `machine01, machine02, machine99` each occupy one level

But, not match:

`factory/machine01/vibration`

→ final level ≠ temperature

`factory/line1/machine01/temperature`

→ too many levels

"+" → don't care what appears at this level

Example: `factory/machine01/+`

Match:

`factory/machine01/temperature`

`factory/machine01/vibration`

`factory/machine01/status`

3. MQTT and Publish-Subscribe

MQTT wildcards – Multi-level “#”

Multi-level wildcard: “#”

- “#” matches zero / more remaining topic levels

Example:

`factory/#`

- **Match:**

  `factory`

  `factory/machine01`

  `factory/machine01/temperature`

  `factory/machine01/vibration`

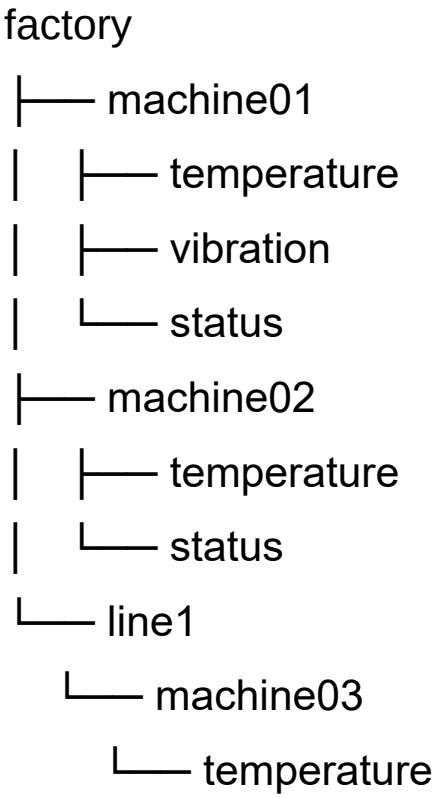
  `factory/line1/machine02/status`

  `factory/buildingA/room3/humidity`

→ Everything under `factory` branch

→ Subscription: `factory/#`

→ Subscribe to whole subtree



3. MQTT and Publish-Subscribe

MQTT wildcards – “+” vs “#”

Subscription	Meaning
`factory/+/temperature`	Temperature from any one machine
`factory/machine01/+`	Any one-level item from machine01
`factory/#`	Everything under factory

- → “+” replaces one level
- → “#” replaces all remaining levels

3. MQTT and Publish-Subscribe

MQTT wildcards – Example

Suppose the broker has:

- `factory/machine01/temperature`
- `factory/machine01/vibration`
- `factory/machine02/temperature`
- `factory/machine02/status`
- `office/room101/temperature`

Subscribe A: `factory/+/temperature`

→ Receives:

- `factory/machine01/temperature`
- `factory/machine02/temperature`

Subscribe B: `factory/machine01/+`

→ Receives:

- `factory/machine01/temperature`
- `factory/machine01/vibration`

Subscribe C: `factory/#`

→ Receives:

- `factory/machine01/temperature`
- `factory/machine01/vibration`
- `factory/machine02/temperature`
- `factory/machine02/status`

3. MQTT and Publish-Subscribe

MQTT wildcards – Importance

Imagine:

`500 machines`

`×`

`3 measurements each`

`=`

`1,500 topics`

A monitoring dashboard only need temperatures

- Without wildcard: `500 separate subscription` (each machine)
- With wildcard: `factory/+/temperature`
- Just **one subscription filter** cover all

Gives MQTT:

- ✓ easier configuration
- ✓ better scalability
- ✓ simpler dashboards
- ✓ easier addition of new devices
- ✓ less application-side subscription management

Example: `machine501` is added later

`factory/machine501/temperature`

- **Dashboard already subscribed to**: `factory/+/temperature`
- Can receive automatically
- No new individual subscription is needed

3. MQTT and Publish-Subscribe

Topic vs payload

Topic is not payload

- Topic:

```
factory/machine01/temperature
```

- Payload:

```
{
  "value": 68.4,
  "unit": "C"
}
```

Topic → message classification / routing

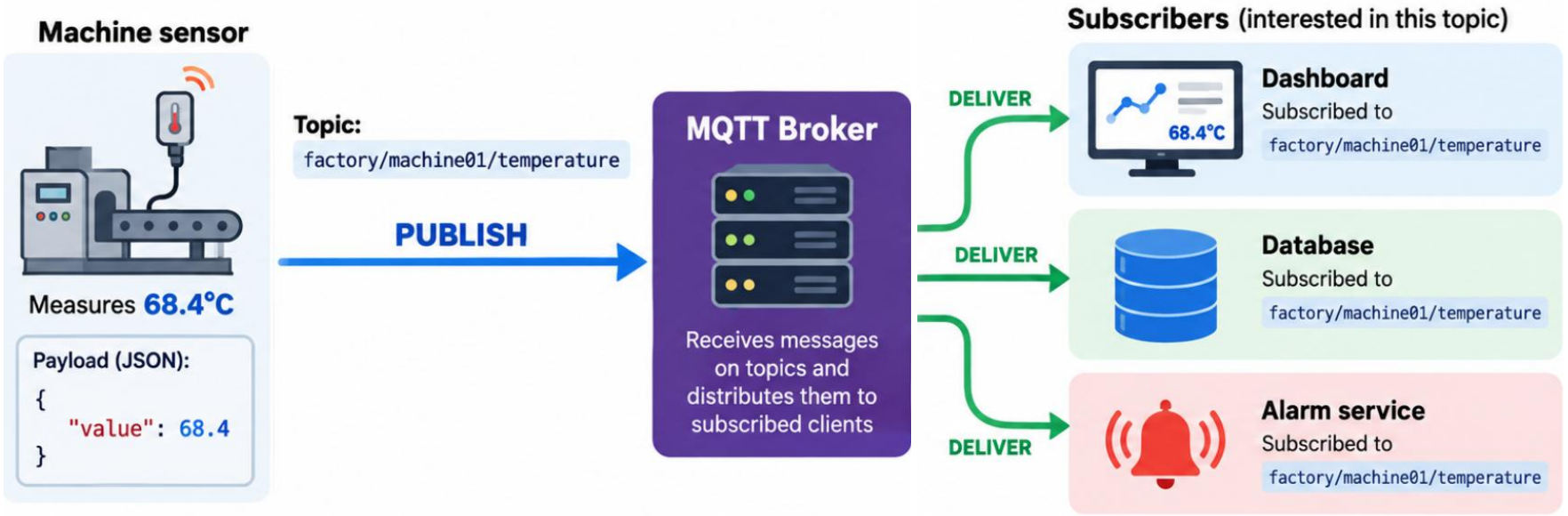
Payload → actual information

→ MQTT does not require JSON

→ Payload – application-defined binary data

3. MQTT and Publish-Subscribe

Complete MQTT message flow



- Sensor knows:
  - ✓ Broker address
  - ✓ Topic
  - ✓ Payload
- Sensor does not necessarily knows:
  - ✓ Dashboard address
  - ✓ Database address
  - ✓ Alarm-system address

Benefits:

- Sensor sends data to broker
- Broker distributes it to interested applications

What Is MQTT and How It Works

https://www.youtube.com/watch?v=EZAhu31jSUg

3. MQTT and Publish-Subscribe

MQTT QoS

What delivery assurance does this MQTT publication require?

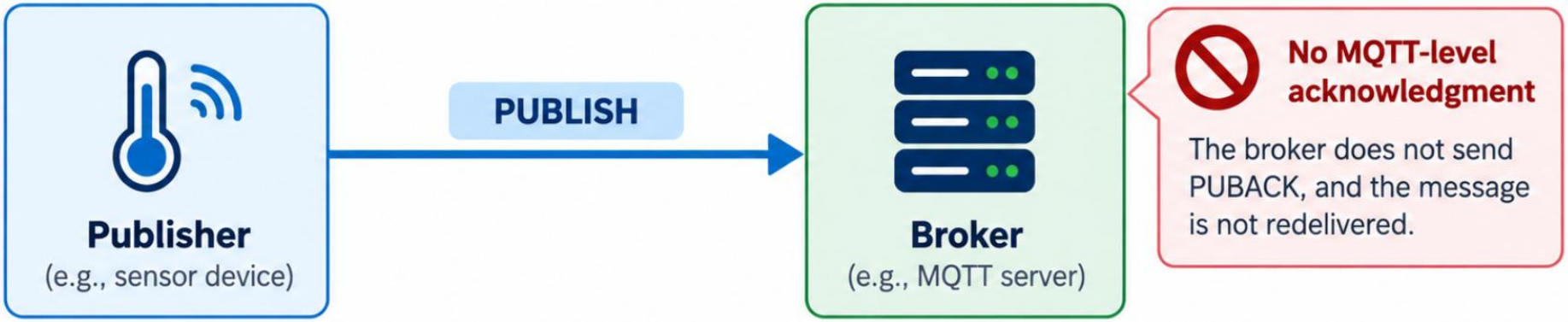
→ MQTT QoS (Quality of Service) levels

- Defines the MQTT-level delivery semantics
  - ✓ To **ensure reliable data delivery** over unstable network
- **IoT networks** may experience frequent drops, latency, and interference
- QoS solves these issues by letting users choose the exact balance between:
  - Reliability
  - Speed
  - Network cost

3. MQTT and Publish-Subscribe

MQTT QoS 0 – At most once

- Publisher sends a PUBLISH message once
- MQTT does not require a publication acknowledgment
- MQTT does not perform MQTT-level redelivery for the message



- Typical use:
  - **Frequent non-critical** telemetry (periodic temperature updates),
    - where next reading will arrive soon

3. MQTT and Publish-Subscribe

MQTT QoS 1 – At least once

- **QoS 1** ensures a published message is delivered **at least once** to the broker
- Publisher waits for an acknowledgment (PUBACK)
- If PUBACK is not received, publisher may retransmit retransmit



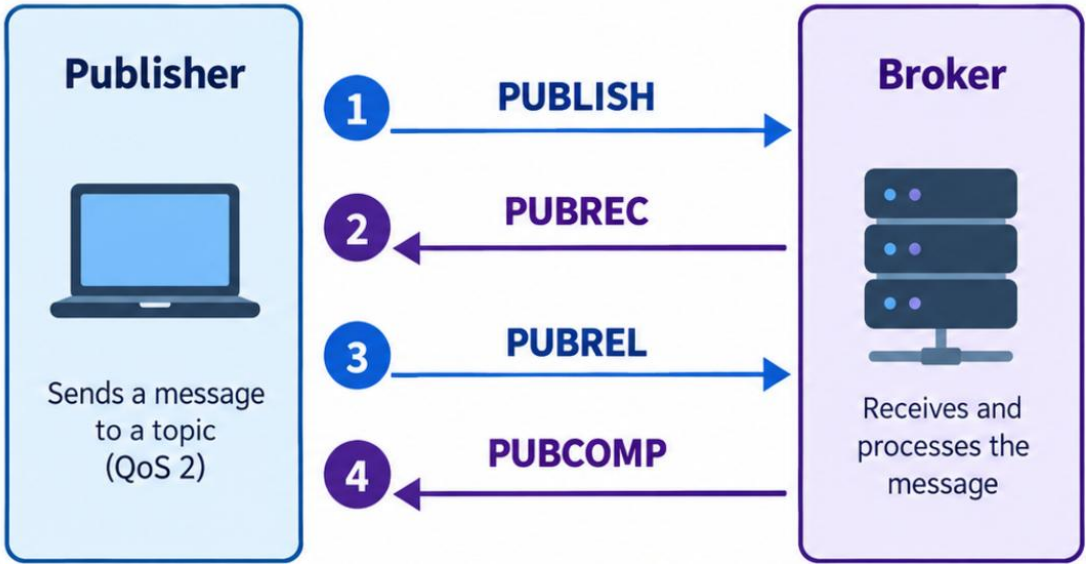
- Due to network issues or broker failure,
  - Publisher might not receive PUBACK and resend same message

    → Duplicate delivery
- Typical use:
  - **Important** telemetry or alerts that should be delivered
    - Even if duplicates are possible

3. MQTT and Publish-Subscribe

MQTT QoS 2 – Exactly once

- **Messages arrive exactly once without duplicates**
- Additional exchanges: PUBLISH, PUBREC, PUBREL and PUBCOMP
- More protocol interaction **provides stronger delivery semantics**
  - But **more overhead**



- Typical use:
  - Commands or transactions where **duplicate processing should be avoided**

3. MQTT and Publish-Subscribe

MQTT QoS

QoS selection:

QoS	Delivery behavior	Best for	Example	Trade-off
0	At most once	Frequent, non-critical data	Temperature telemetry	Lowest overhead, possible loss
1	At least once	Important messages	Fire alarm Machine fault alarm	More reliable, duplicates possible
2	Exactly once*	Messages where duplicate MQTT delivery should be avoided	Critical control message Financial transaction	Highest overhead

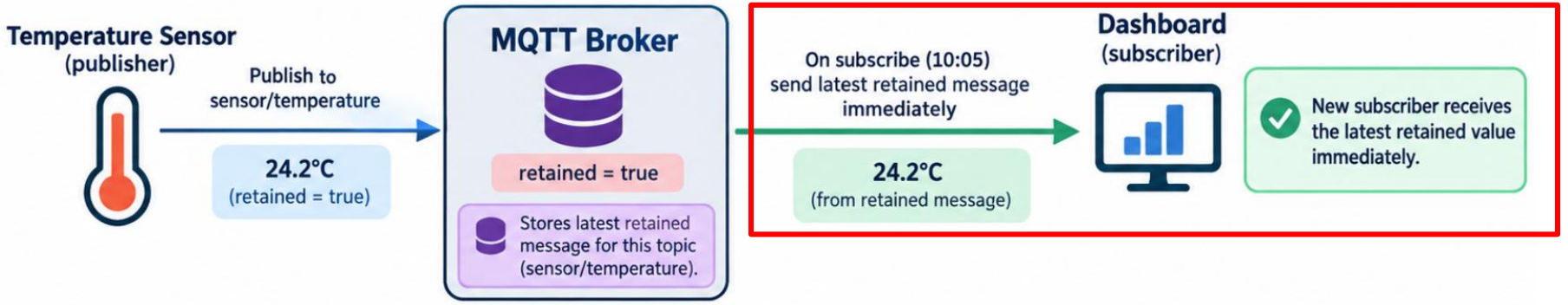
3. MQTT and Publish-Subscribe

MQTT Retained message

- Solves new-subscriber problem by storing and delivering latest message to new subscribers
- Example:



- **Retained message** = **latest retained state**, not complete history
- **Broker** stores only most recent retained message for each topic and delivers to new subscribers

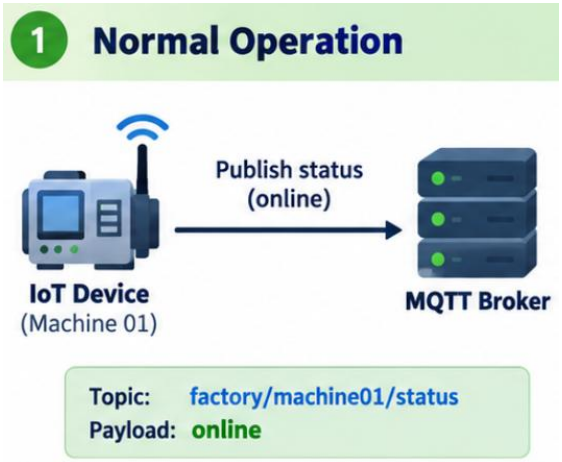


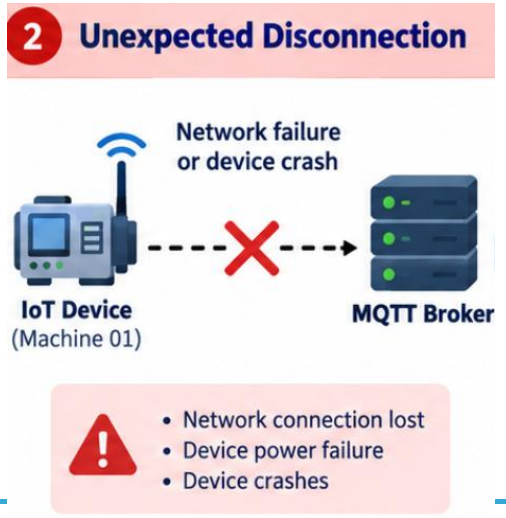
3. MQTT and Publish-Subscribe

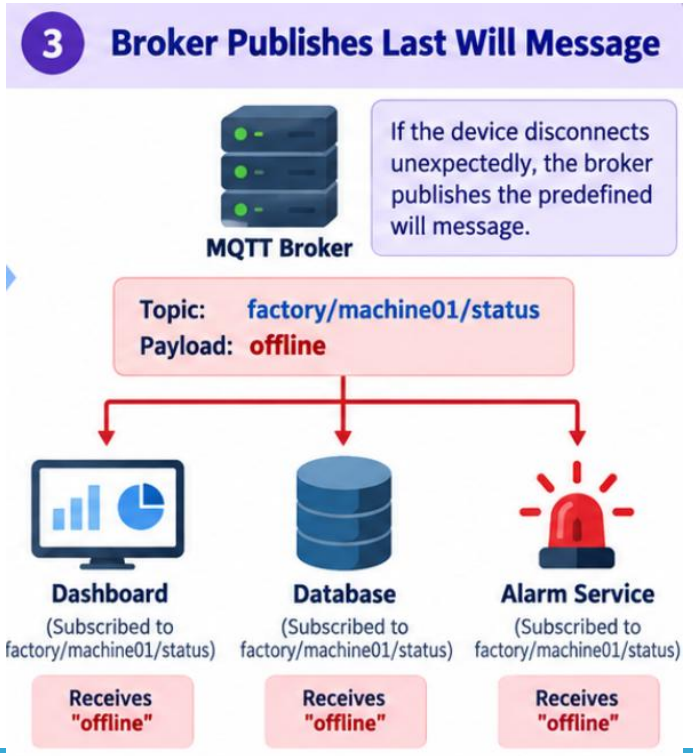
MQTT Last Will

- **Last Will** – predefined emergency message
  - Defined in advance when the client connects to the broker
  - Published only if the client disconnects unexpectedly
- Example:
  - Topic: `factory/machine01/status`
  - Payload: `offline`
  - So if the device crashes or loses network, the broker publishes:

    `factory/machine01/status = offline`







3. MQTT and Publish-Subscribe

Comparison

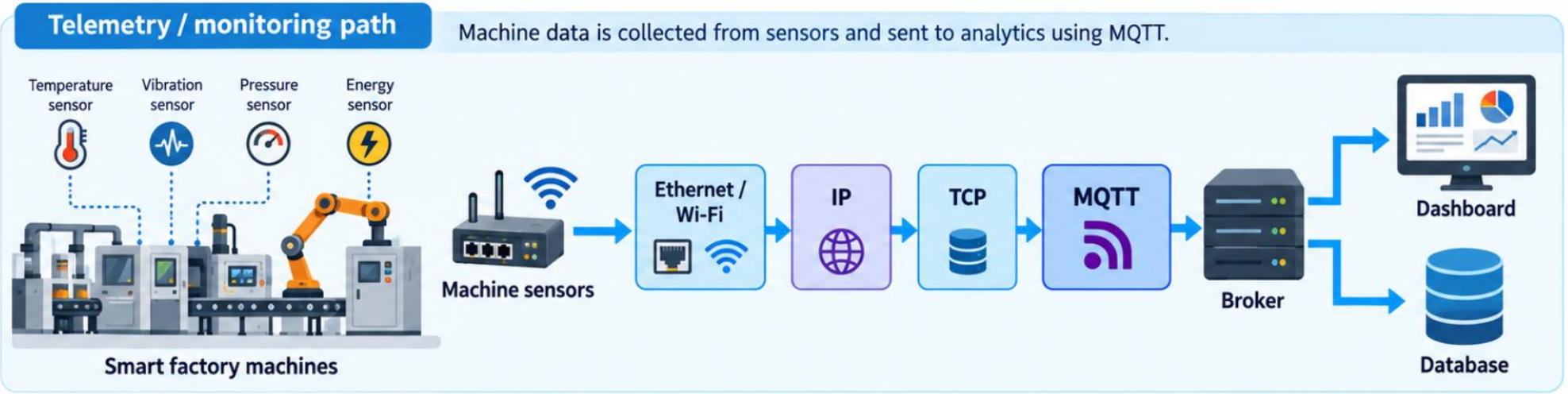
Question	HTTP	CoAP	MQTT
Main model	Request–response	Request–response	Publish–subscribe
Common transport	TCP	UDP	TCP
Architecture	Client–server	Client–server/resource	Broker based
Strength	Web/API integration	Constrained resources	Telemetry/event distribution
Typical IoT example	Cloud REST API	Constrained sensor	Continuous sensor messaging

- They are not necessarily mutually exclusive
- A real IoT system may use more than one

3. MQTT and Publish-Subscribe

MQTT in smart factory system

- **MQTT for telemetry**



Why use MQTT instead of HTTP in IoT applications?

https://www.youtube.com/watch?v=bmBN_pP5rWU

Using Node RED & MQTT with PLCnext Technology

https://www.youtube.com/watch?v=Hqlot2xC4Ng

3. MQTT and Publish-Subscribe

Trace message from sensor to cloud

Week 3

Physical temperature

↓

Sensor reading

↓

Structured data

↓

JSON payload

Week 4

Wi-Fi connectivity

↓

Access Point

IP

↓

TCP

↓

MQTT

↓

Topic

factory/machine01/temperature

↓

Broker

↓

Dashboard

- What does Wi-Fi solve?
  - Local connectivity
- What does IP solve?
  - Network addressing/delivery
- What does TCP solve?
  - Reliable transport stream
- What does MQTT solve?
  - Application messaging and publish–subscribe distribution
- What does JSON solve?
  - Data representation

3. MQTT and Publish-Subscribe

Selecting communication approach

1. What interaction is needed?
   a. Read/change a resource? → request-response
   b. Continuous telemetry/events? → publish-subscribe
2. Is the endpoint constrained?
   a. Very constrained device/network? → consider CoAP
   b. Web/cloud integration? → HTTP
   c. Brokered telemetry? → MQTT
3. What delivery behavior does the application require?

Requirement	**Mechanism to consider**
Reliable, ordered transport	**TCP**
Lightweight transport without built-in reliability	**UDP**
MQTT publication delivery assurance	**MQTT QoS 0/1/2**

Summary

APPLICATION Layer HTTP | CoAP | MQTT	Where application-specific data is generated
↓	
TRANSPORT Layer TCP | UDP	Manages end-to-end process-to-process communication using source and destination port numbers
↓	
NETWORK Layer IPv4 | IPv6	Provides host-to-host addressing and internetwork routing
↓	
ADAPTATION where needed 6LoWPAN	Positioned directly between the Network layer and constrained physical links
↓	
CONNECTIVITY Ethernet | Wi-Fi | 802.15.4 | Cellular | ...	Physical and data link layers

Checkpoint

“Suppose a factory temperature sensor publishes telemetry through Wi-Fi to an MQTT broker”

MQTT

↓

TCP

↓

IP

↓

Wi-Fi

“What does each layer solve?

- Wi-Fi gives connectivity
- IP reaches the broker's host
- TCP provides reliable transport between applications
- MQTT provides publish–subscribe messaging

Checkpoint

“Battery-powered room sensor. IEEE 802.15.4. Very small messages. We want to request its temperature resource.”

CoAP

↓

UDP

↓

IPv6

↓

6LoWPAN

↓

IEEE 802.15.4

- **Constrained device and link** have different resource and communication characteristics